Production Plan — *Clink* Clinic & Appointments Web App

> **Prepared by:** Senior Full-Stack Architect & DevOps Engineer
> **Date:** 2026-06-28
> **Scope:** Production launch plan for desktop + mobile web (PWA) of the Django clinic platform.
> **Priority directive:** *Best price for value* across every category, for a budget-conscious solo / small-team founder in the Palestine / MENA market.

Stack analyzed: Django 6.0.1 · Gunicorn (WSGI) · PostgreSQL · Redis (django-redis) · WhiteNoise · server-rendered templates + HTMX partials · DRF + SimpleJWT · Arabic-RTL + English (i18n) · PHI / healthcare data · SMS via TweetsMS + Twilio Verify · email via Brevo. Currently: a `.COM` test domain at Hostinger; no PWA, no CI/CD, migrations git-ignored, media on local disk.

> ⚠️ **On pricing:** all dollar/euro figures are **2026 approximations** gathered during research — treat them as planning estimates and **verify at signup**. Intro VPS rates differ from renewal rates.

Table of Contents

Executive Summary

Your "clink app" is in a strong position to ship cheaply and safely: a server-rendered Django 6 + HTMX + WhiteNoise + Postgres + Redis stack with no Node build step, and an unusually mature auth/security posture (TOTP MFA with Fernet-encrypted secrets, Redis-backed brute-force lockouts, PHI export guards,

`check --deploy` gating). The recommended production posture at a glance: a single low-cost VPS (Hetzner CX22 in Frankfurt — lowest latency to Gaza) running Caddy + Gunicorn + Postgres + Redis, with **Cloudflare Free** in front of everything (DNS, TLS edge, WAF, CDN, cookieless analytics), patient media moved to **Cloudflare R2** (private buckets, signed URLs, zero egress), and a free observability stack (Sentry Developer, Better Stack Uptime, Cloudflare Web Analytics). The PWA is pure hand-written static files plus your existing TLS — effectively $0.

The single most important decisions are not about hosting — they are three repo-hygiene fixes that gate everything else. (1) **Un-ignore and commit all migrations** (`.gitignore` line 18 blanket-ignores them); without this, deploys and restores are non-reproducible and a PHI database cannot be rebuilt. (2) **Move media off local disk** (`MEDIA_ROOT = BASE_DIR/media` , confirmed at settings.py:254) to private R2 with short-lived signed URLs — today every redeploy silently destroys prescription/lab-scan uploads and blocks horizontal scaling. (3) **Add a `/healthz` endpoint** (confirmed absent) so uptime monitoring and zero-downtime deploys are even possible. Do these before go-live; the rest of the plan assumes them.

Two security fixes are equally cheap and high-impact: bind Gunicorn to `127.0.0.1` so the unconditional `SECURE_PROXY_SSL_HEADER` (settings.py:324) can't be spoofed, and turn on JWT `ROTATE_REFRESH_TOKENS` + `BLACKLIST_AFTER_ROTATION` (both confirmed `False` at settings.py:280-281) with the blacklist app installed, so a stolen 1-day refresh token is revocable.

Headline monthly cost: **Launch ≈ $5–8/mo** in fixed infra (one Hetzner CX22 VPS + free Cloudflare/R2/Sentry/Better Stack tiers), plus pay-per-use SMS — which will be your single largest and most variable expense (~$60–150/mo at modest reminder volume on MENA SMS rates, controllable via email/WhatsApp opt-in). **Growth ≈ $55–110/mo** fixed as you add managed Postgres + PITR, a managed Redis, an async worker (Django-Q2), Cloudflare Pro, and Sentry Team. The architecture scales vertically for a long time on one box, and horizontally for free once media is on R2 — no SPA rewrite required, exactly as constrained.

Recommended Stack at a Glance

Category	Recommended pick	Why	Approx $/mo (Launch)
Web/app hosting	Hetzner Cloud CX22, Frankfurt (Caddy + Gunicorn)	Lowest latency to Gaza (~60–80 ms); best price/spec; one box runs everything. Fallback: Hostinger VPS if Hetzner signup is rejected	~€4.49 + €0.50 IPv4 (≈ $5)
Database	Postgres 16 local on the VPS (least-priv app role)	Co-located = $0 extra at MVP; LUKS disk encryption for PHI at rest	$0 (on VPS)
Redis	Redis 7 local on the VPS	Backs cache + OTP throttle + brute-force lockouts + MFA limits	$0 (on VPS)
Object storage (media/PHI)	Cloudflare R2, private bucket, signed URLs	Fixes ephemeral-media loss; 10 GB free; **zero egress** = no bandwidth bill on PHI downloads	~$0 (under 10 GB)

Category	Recommended pick	Why	Approx $/mo (Launch)
CDN / edge	Cloudflare Free (proxied)	Edge-caches WhiteNoise hashed static + fonts near MENA; DDoS; Full(strict) TLS	$0
DNS	Cloudflare Free DNS	Anycast, CNAME flattening at apex, zone export makes brand cutover trivial	$0
TLS / WAF	Caddy (origin, auto Let's Encrypt) + Cloudflare WAF Free	3-line Caddyfile auto-renews; CF Bot Fight Mode + challenge on login/OTP/ `/api/token/`	$0
Transactional email	Brevo (Sendinblue) free tier + DKIM/DMARC	300 emails/day shared; DKIM is the deliverability must-have	$0 (pay-as-you-grow)
SMS / OTP	TweetsMS primary + Twilio Verify fallback	In-region cost via TweetsMS, automatic failover to Twilio	usage (see cost table)
CI	GitHub Actions	2,000 free min/mo; pipeline runs 2–4 min; guards committed migrations	$0
Error tracking	Sentry Developer (`send_default_pii=False` + PHI scrub)	5k events/mo free; auto-instruments Django/Redis/DB	$0
Uptime	Better Stack Uptime free	Commercial-OK (unlike UptimeRobot free), 10 monitors + status page	$0
Analytics (RUM/CWV)	Cloudflare Web Analytics	Cookieless → no Arabic consent banner to build	$0
Backups / DR	Nightly `pg_dump | gpg` → R2 + R2 object versioning	Encrypted off-box copy; managed PITR added at Growth	~$0 (within R2 free)

---

1. PWA & Frontend Architecture

This app is a server-rendered Django + HTMX + WhiteNoise stack with **no Node build step** — which is actually ideal for a lean PWA. You don't need Workbox tooling, npm, or a bundler. Everything below is hand-written, version-controlled, served by WhiteNoise, and costs **$0/month** (PWA infra is just static files + your existing TLS). The only real spend is optional Web Push, covered at the end.

> **Stack-specific gotcha up front:** your `migrations` are `.gitignore`d and `MEDIA_ROOT` is local disk. Those bite *backend* reproducibility, but they also touch you here: **do not** let the service worker cache anything under `MEDIA_URL` (patient-uploaded files / PHI) and do not cache hashed static if the manifest isn't committed. Treat the SW cache as "public, non-PHI, static only."

1.1 Adding PWA capability with no bundler

Three hand-written files + two `<link>` / `<script>` tags. All live in your **static root** except where scope forces otherwise (see the SW caveat).

`static/manifest.webmanifest` (serve as a static file; WhiteNoise handles it):

```
{
  "name": "Clink – Clinic & Appointments",
  "short_name": "Clink",
  "lang": "ar",
  "dir": "rtl",
  "start_url": "/?source=pwa",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait",
  "background_color": "#ffffff",
  "theme_color": "#0d9488",
  "icons": [
    { "src": "/static/icons/icon-192.png", "type": "image/png", "sizes": "192x192" },
    { "src": "/static/icons/icon-512.png", "type": "image/png", "sizes": "512x512" },
    { "src": "/static/icons/maskable-512.png", "type": "image/png", "sizes": "512x512", "purpose":
"maskable" }
  ]
}
```

In **every base template** (`accounts/base` , `doctors/base_doctor` , `patients/base_dashboard` , `secretary/base_secretary`) add to `<head>` :

```
<link rel="manifest" href="{% static 'manifest.webmanifest' %}">
<meta name="theme-color" content="#0d9488">
<link rel="apple-touch-icon" href="{% static 'icons/apple-touch-icon-180.png' %}">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="default">
<meta name="apple-mobile-web-app-title" content="Clink">
```

The service-worker scope trap (critical for this stack): a SW can only control pages **at or below its own URL path**. WhiteNoise serves your JS from `/static/…` , so a SW at `/static/sw.js` would only control `/static/*` — useless. You need the SW served from **site root** (`/sw.js`) with `Service-Worker-Allowed: /` . Two clean options:

- **Recommended:** add a tiny Django view/URL that returns the SW file with the right headers — keeps it in your repo, lets you template the cache version, and guarantees the MIME type:

```
# clinic_website/urls.py
from django.views.generic import TemplateView
urlpatterns += [
    path("sw.js", TemplateView.as_view(
        template_name="sw.js",
        content_type="application/javascript",
    )),
    path("offline/", TemplateView.as_view(template_name="offline.html")),
]
```

Put `sw.js` and `offline.html` in a templates dir. The root URL gives you `scope: "/"` for free (no special header needed when the file is already at root).

- Alternative: serve `sw.js` as static and set `Service-Worker-Allowed: /` via WhiteNoise `WHITENOISE_ADD_HEADERS_FUNCTION`. More fiddly; the URL view is simpler.

Registration snippet (one shared JS file, deferred, in your base templates):

```
<script>
if ("serviceWorker" in navigator) {
  window.addEventListener("load", () =>
    navigator.serviceWorker.register("/sw.js", { scope: "/" }));
}
</script>
```

SWs require a **secure context**: HTTPS in prod (you already force `SECURE_SSL_REDIRECT` + HSTS) and `localhost` for dev. Your test `.COM` on Hostinger over HTTPS works today; the final brand domain will too.

1.2 Offline strategy that fits a PHI app

For a healthcare app, the SW is a **performance + resilience** tool, **not** an offline-data tool. The rule: **cache the shell and public static; never cache PHI, authenticated HTML, or any non-GET.**

Recommendation: **~100-line vanilla SW, not Workbox.** Rationale for *this* stack — Workbox's value is its build-time precache manifest generation, which assumes a Node build you deliberately don't have. Pulling Workbox from a CDN at runtime adds a third-party dependency and ~20KB for caching logic you can write in 80 lines. Vanilla also makes the "don't cache PHI" guard explicit and auditable (matters for your `compliance` app).

```
// sw.js  (templated by Django so {{ CACHE_VERSION }} busts on deploy)
const VERSION = "{{ CACHE_VERSION }}";          // e.g. git SHA or settings value
const SHELL = `shell-${VERSION}`;
const STATIC = `static-${VERSION}`;
const PRECACHE = ["/offline/", "/static/css/app.css", "/static/icons/icon-192.png"];

self.addEventListener("install", (e) => {
  e.waitUntil(caches.open(SHELL).then((c) => c.addAll(PRECACHE)));
  self.skipWaiting();                           // activate new SW immediately
});

self.addEventListener("activate", (e) => {
  e.waitUntil(
    caches.keys().then((keys) =>
      Promise.all(keys.filter((k) => !k.endsWith(VERSION)).map((k) => caches.delete(k))))
  );
  self.clients.claim();                         // take control of open tabs
});

self.addEventListener("fetch", (e) => {
  const { request } = e;
  const url = new URL(request.url);

  // HARD GUARDS — never touch PHI / auth / mutations / cross-origin.
  if (request.method !== "GET") return;                       // no POST/PUT
  if (url.origin !== self.location.origin) return;            // no CDN/PHI hosts
  if (url.pathname.startsWith("/media/")) return;             // patient uploads = PHI
  if (request.headers.get("Authorization")) return;           // DRF/JWT calls
  if (request.headers.get("HX-Request")) return;              // HTMX partials = live data

  // Static assets (hashed by WhiteNoise) → stale-while-revalidate.
  if (url.pathname.startsWith("/static/")) {
    e.respondWith(
      caches.open(STATIC).then(async (cache) => {
        const cached = await cache.match(request);
        const network = fetch(request).then((res) => {
          if (res.ok) cache.put(request, res.clone());
          return res;
        });
        return cached || network;
      })
    );
    return;
  }

  // Top-level navigations → network-first, fall back to offline page.
  // We DO NOT cache the HTML response (it may contain PHI for logged-in users).
  if (request.mode === "navigate") {
    e.respondWith(fetch(request).catch(() => caches.match("/offline/")));
  }
});
```

Why each guard matters here: `HX-Request` skip protects your `ws_notes` / `ws_orders` / `ws_prescriptions` partial swaps (live clinical data); `/media/` skip protects uploaded scans/lab files; `Authorization` skip protects the DRF/JWT surface. Cache versioning + `skipWaiting()` + `clients.claim()` ensure a deploy invalidates the old shell instead of stranding users on stale assets — important because your static is content-hashed and immutable-cached.

1.3 Installability + app-like UX

Asset	Size / spec	Notes
Standard icon	192 & 512 PNG	required for Android install
Maskable icon	512, `purpose:maskable`	with safe-zone padding, else Android crops it
`apple-touch-icon`	180×180 PNG	iOS home-screen icon (manifest icons ignored by iOS)
Splash	derived from theme/bg	iOS auto-generates from `theme_color` + icon

iOS/Safari quirks you must plan around: - **No `beforeinstallprompt` on iOS.** You cannot trigger an install programmatically. Show a one-time RTL hint ("للتثبيت: شارك ← أضف إلى الشاشة الرئيسية") for iOS Safari users instead of an install button. - iOS uses the `apple-mobile-web-app-*` meta tags above, not the manifest, for standalone behavior and title. - On Android/desktop Chrome, you *can* capture `beforeinstallprompt` and show a custom "تثبيت التطبيق" button — worth doing since it lifts install rate.

1.4 Push notifications — feasibility & timing

Verdict: defer to post-launch. You already have SMS (TweetsMS + Twilio Verify) and email (Brevo) — those cover appointment reminders today with no new moving parts. Web Push adds VAPID key management, a `PushSubscription` store, and a sender. Specifics for your market:

- iOS web push **requires the PWA be installed to the Home Screen** (iOS 16.4+); it does **not** work from a Safari tab. Each step (install → grant permission) sheds audience, so iOS reach is low until users install. (magicbell, batch)
- Android/desktop Chrome web push works from the tab — easier, but your reminder use-case is already served by SMS.

When you do add it (Growth tier), do it **self-hosted, not via a SaaS** — you control PHI and avoid per-MAU fees:

Option	Monthly cost	Verdict
Self-host Web Push (VAPID) — `pywebpush` lib + generated VAPID keys, store subscriptions in Postgres, send from a Celery/cron task	**$0** (uses your Gunicorn/Redis already)	**RECOMMENDED** — no per-message fee, PHI never leaves your infra
OneSignal / Pushengage free tier	$0 up to a cap, then ~$9–$99/mo	Avoid: routes notification metadata through a third party — extra BAA/compliance surface for PHI

Keep push payloads **PHI-free** ("لديك تحديث في موعدك" + a link, never the diagnosis) regardless.

1.5 Frontend performance (desktop + mobile, MENA networks)

Your biggest, cheapest win is **killing the three render-blocking third-party CDN requests in `<head>`** (Google Fonts CSS, the font files, Font Awesome). On a 4G MENA connection these are your top LCP

killers. Self-host all of it through WhiteNoise.

Fonts (Cairo + Inter): 1. Pull WOFF2 from **google-webfonts-helper** (`gwfh.mranftl.com`), selecting **`arabic`** + **`latin`** subsets only — drop cyrillic/vietnamese/etc. (gwfh, github) Cairo's full family is large; subsetting to arabic+latin and only the **weights you actually use** (e.g. 400/600/700) cuts hundreds of KB. 2. Drop the woff2 in `static/fonts/` , write your own `@font-face` with **`font-display: swap`** . 3. **Preload** the two critical faces (Arabic 400/600, since `ar` is your default) to fix the LCP text:

```
<link rel="preload" as="font" type="font/woff2"
      href="{% static 'fonts/cairo-arabic-400.woff2' %}" crossorigin>
```

Font Awesome: you almost certainly use a handful of icons. Self-hosting the full kit is wasteful. **Recommended: replace with an inline SVG sprite of only-used icons** (a single `static/icons/sprite.svg` , referenced via `<use>`), eliminating an entire CSS file + webfont. If that's too much churn pre-launch, at minimum **self-host FA's webfont subset** via WhiteNoise rather than the CDN.

WhiteNoise config — turn on hashed + compressed immutable caching (you have `whitenoise 6.11`):

```
# settings.py
STORAGES = {
  "staticfiles": {"BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage"},
}
```

Then `pip install whitenoise[brotli]` so WhiteNoise serves **brotli** to HTTPS clients and gzip otherwise; hashed filenames get `Cache-Control: immutable` for a year. (whitenoise docs) This pairs perfectly with the SW's stale-while-revalidate static caching above.

Other concrete fixes: - **Defer JS:** add `defer` to your scripts; the SW registration and HTMX should not block parse. - **Critical CSS:** inline the above-the-fold shell CSS (header + bottom-nav) in `<head>` , load the rest of `app.css` normally. With hand-written CSS this is a manual copy — but high-leverage for LCP. - **CLS:** put explicit `width` / `height` (or `aspect-ratio`) on every `<img>` (doctor avatars, logos, uploaded thumbnails) and reserve space for the glass bottom-nav so it doesn't shift content. `font-display:swap` + preload prevents the late-font reflow. - **Images via Pillow:** you already depend on `pillow` — generate responsive sizes + **WebP** on upload (AVIF optional), serve with `srcset` / `sizes` + `loading="lazy"` + `decoding="async"` for off-screen images. *(But remember: uploaded images live under `/media/` — fix the ephemeral-storage risk separately; that's an infra-section concern.)*

1.6 Core Web Vitals targets + mobile budget

Metric	Target (mobile, 4G MENA)
LCP	< 2.5 s
INP	< 200 ms
CLS	< 0.1

Metric	Target (mobile, 4G MENA)
HTML (gz)	< 30 KB per role page
CSS (br)	< 60 KB total
Fonts	≤ 2 woff2 preloaded, < 120 KB combined
JS	< 50 KB (HTMX ~14 KB + SW reg + small glue)
Requests in `<head>`	**0 third-party** after self-hosting

HTMX naturally keeps payloads small (partial swaps), so this budget is realistic without a bundler.

1.7 RTL / i18n + accessibility

- Verify `dir="rtl"` + `lang="ar"` on `<html>` for the default locale and that it flips to `ltr` / `en` correctly under `i18n`. Use CSS **logical properties** (`margin-inline-start` , `inset-inline-end` , `padding-block`) throughout so one stylesheet serves both directions — critical for your glass bottom-nav and desktop layout.
- Manifest must carry `dir: "rtl"` , `lang: "ar"` (above) so the install UI renders correctly.
- Accessibility that also helps CWV/SEO: semantic landmarks, real `<button>` / `<a>` , and proper focus management. You **already ship accessible modals** (focus trap + Escape on the cancel-appointment and rating modals) — extend that same pattern to any HTMX-swapped dialog and to the install prompt.

1.8 Do this now vs later

Now (pre-launch, all $0, no build step): 1. Self-host fonts (arabic+latin subset, preload, `font-display:swap`) and remove the Google Fonts CDN. *(Biggest LCP win.)* 2. Turn on `CompressedManifestStaticFilesStorage` + `whitenoise[brotli]` . 3. Add `manifest.webmanifest` , icons (incl. maskable + apple-touch), `theme-color` , iOS meta tags → installable. 4. Ship the vanilla SW at `/sw.js` (Django URL) with the PHI guards + `/offline/` page; version it by deploy. 5. CLS pass: `width` / `height` on all images, reserve bottom-nav space, `defer` JS.

Later (Growth): 6. Replace Font Awesome with an inline SVG sprite of used icons. 7. Pillow upload pipeline → WebP responsive `srcset` + lazy-load. 8. Custom Android/desktop install prompt via `beforeinstallprompt` . 9. Self-hosted Web Push (VAPID + `pywebpush`), PHI-free payloads — only after install rates justify it. 10. *Optional* lightweight build only if asset count grows: a single `esbuild` / `lightningcss` binary call in your Render build command (no `package.json` needed) for minify+critical-CSS extraction. Skip until manual maintenance actually hurts.

Sources: WhiteNoise docs · google-webfonts-helper · Cairo on gwfh · iOS PWA push limitations (magicbell) · iOS home-screen push requirement (Batch)

---

2. Hosting & Infrastructure

The settings already assume a **TLS-terminating reverse proxy** in front of Gunicorn (`SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")` + `SECURE_SSL_REDIRECT` + HSTS preload). That single fact drives the whole topology: whatever you run, Gunicorn must sit **behind** something that terminates HTTPS and sets `X-Forwarded-Proto: https`. Both options below satisfy that — a PaaS does it for you; on a VPS you run Caddy or Nginx.

A second hard constraint: you have **5 stateful dependencies** — Postgres, Redis, DB-backed sessions, local media, and WhiteNoise statics. Four of those are already external-ready. **Media on local disk is the one blocker to horizontal scaling and the one that silently destroys patient uploads on every redeploy** — fixed below.

2.1 Where to host — comparison (MVP scale)

Option	~Monthly (MVP)	Persistent disk?	Ops burden	MENA latency	Pay-friendly for PS founder	Verdict
Hetzner Cloud CX22 (Frankfurt, 2 vCPU / 4 GB / 40 GB)	**~€4.49 + €0.50 IPv4 ≈ $5** (media on free R2)	Yes — full root disk	Medium (you run Caddy/Postgres/Redis)	**Best — Frankfurt ~60–80 ms to Gaza**	Card/PayPal; **may reject some MENA signups — create the account first; Hostinger is the fallback**	⭐ **RECOMMENDED (your pick)**
Hostinger VPS KVM 2 (2 vCPU / 8 GB / 100 GB NVMe)	**~$7–9** intro, ~$14 renew	**Yes — full root disk**	Medium	Good (EU DCs)	**Same account as your domain — zero signup friction**	**Fallback** if Hetzner rejects your region/payment
Render (Web Starter $7 + PG $7–20 + Redis $10)	**~$24–37**	**No — ephemeral FS** (must use object storage)	**Lowest** (matches your `DEPLOY_RENDER.md`)	US/EU/Singapore regions; pick **Frankfurt**	Card; works	Good if you value zero-ops over price
Railway (Hobby $5 + usage)	~$15–30	No (volumes extra)	Low	EU metal available	Card	Fine; pricing drifts with usage

Option	~Monthly (MVP)	Persistent disk?	Ops burden	MENA latency	Pay-friendly for PS founder	Verdict
Fly.io	~$30+ (Managed PG from **$38**)	Volumes yes	Low-med	**No nearby region; PG got pricey**	Card	Skip — DB pricing kills the value
DigitalOcean droplet	~$6–12	Yes	Medium	AMS/FRA ok	Card	Fine but pricier than Hetzner for same specs

Recommendation for a solo, budget-first MENA founder:

- **Go with Hetzner Cloud CX22 (Frankfurt, ~€4.49/mo + ~€0.50 IPv4 ≈ $5).** One box runs *everything* — Caddy + Gunicorn + Postgres + Redis — on a persistent root disk, with the lowest latency to Gaza (~60–80 ms) and the best price-to-spec here. Media goes to **Cloudflare R2 (free)**, so you do *not* need Hetzner's paid object storage.
- **Do this first:** create the Hetzner account and add a payment method (card or PayPal) *before* you build anything. Hetzner has occasionally rejected MENA signups — you want to learn that on day one, not at launch. If it rejects you, fall straight back to **Hostinger VPS KVM 2 (~$7–9/mo)** — same login as your domain, no new vendor — and every other step in this plan is identical.
- **RAM caveat (important):** CX22 has **4 GB**. Postgres + Redis + 5 Gunicorn workers + your pandas/numpy/scikit-learn analytics can get tight under load. Two cheap fixes: (1) add a **2–4 GB swap file** (one-time, 5 minutes) and keep `--max-requests` worker recycling on; or (2) for ~€2 more, take **Hetzner CAX21 (ARM: 4 vCPU / 8 GB / 80 GB, ~€6.49/mo)** — all your Python deps ship arm64 wheels, so it "just works" and gives real headroom. Recommended if budget allows.
- Skip Render/Railway unless you'd rather pay ~$25–35/mo to never manage a server — their ephemeral disks *force* the R2 media fix anyway (which you're doing regardless).

2.2 Recommended topology (single-VPS MVP — Hetzner CX22)

```
Internet ──TLS──> Caddy (:443, auto Let's Encrypt) ──proxy──> Gunicorn (:8000, unix socket)
                    │  sets X-Forwarded-Proto: https  (matches SECURE_PROXY_SSL_HEADER)
                    ▼
        Postgres 16 (local, port 5432)  +  Redis 7 (local, port 6379)
        Media ──────────────────────────> Cloudflare R2 (private bucket, signed URLs)
        Static ─────────────────────────> WhiteNoise (in-process)
```

Caddy over Nginx here — its `Caddyfile` is ~5 lines, auto-renews TLS, and reverse-proxies to Gunicorn with the correct forwarded headers out of the box:

```
clinic.example.com {
    encode zstd gzip
    reverse_proxy unix//run/gunicorn.sock {
        header_up X-Forwarded-Proto https
    }
}
```

Gunicorn worker count. Use the standard formula and the **sync** worker (your views are blocking template renders, not async):

```
workers = (2 × vCPU) + 1     # CX22 = 2 vCPU → 5 workers
```

`gunicorn clinic_website.wsgi:application --workers 5 --timeout 30 --max-requests 1000 --max-requests-jitter 100 --bind unix:/run/gunicorn.sock` (`--max-requests` recycles workers to cap the pandas/numpy/scikit-learn memory creep in your doctor-analytics code.)

Postgres connection pooling. With 5 Gunicorn workers each holding a connection, set `CONN_MAX_AGE = 60` (persistent conns, not per-request reconnect) but cap it so idle clinics free connections. At MVP one box, that's enough. **At horizontal scale add PgBouncer** in `transaction` pooling mode in front of Postgres and point Django at it — multiple Gunicorn instances × 5 workers will exhaust Postgres's default 100-connection limit fast. Set `pool_mode = transaction` , `default_pool_size = 20` .

Redis stays local (or managed at scale). It already backs your cache, OTP throttling, brute-force lockouts and MFA rate-limits — keep `django-redis` pointed at `redis://127.0.0.1:6379/1` . Note your **sessions are DB-backed**, which is *good* for horizontal scaling (no sticky sessions needed) — leave them in `django_session` .

2.3 Background jobs — your OTP/SMS/email path

Right now this is synchronous and it's a latency + reliability risk. Every TweetsMS SMS, Twilio Verify call, and Brevo email is sent **inside the request/response cycle** — a slow TweetsMS API call blocks the Gunicorn worker and the patient's page hangs; a transient failure becomes a 500 on a login/OTP flow.

- **MVP (keep it cheap):** stay synchronous **but** wrap every outbound provider call with a short timeout (`requests` `timeout=5`) and a try/except that degrades gracefully (e.g. fall back from TweetsMS to Twilio Verify). This is acceptable for launch volume.
- **First scale step (recommended): add Django-Q2 or RQ, not Celery.** You already run Redis — **RQ** (`django-rq`) or **Django-Q2** gives you async SMS/email with ~zero new infra: one extra worker process (`python manage.py rqworker` or `qcluster`) on the same box. Celery is heavier (broker + beat + result backend) than a solo founder needs here. Push SMS/email/Brevo sends to a queue; the OTP *verification* stays sync, only *delivery* goes async.
- Run the worker as a second `systemd` unit (VPS) or a second Render "Background Worker" service (~$7/mo) — only add that cost when send volume justifies it.

2.4 The media-storage fix (do this before launch)

`MEDIA_ROOT = BASE_DIR/media` is **ephemeral on Render/containers and un-shareable across multiple VPS instances** — prescriptions, lab-order scans, and ID uploads vanish on redeploy. Move user uploads to S3-compatible object storage via **`django-storages[s3]`** (boto3 backend).

Provider	Storage	Egress	~10 GB PHI/mo	Notes
Cloudflare R2	$0.015/GB	**$0 (zero egress, all tiers)**	**~$0 (10 GB free tier)**	⭐ **RECOMMENDED** — free tier covers MVP, no egress bill ever, pairs with CF in front
Backblaze B2	$6/TB	free 3× storage, then $0.01/GB; **free via Cloudflare**	~$0 (10 GB free)	Great #2; pair with Cloudflare for free egress
Hetzner Object Storage	€4.99 (incl. 1 TB + 1 TB egress)	€1/TB over	~€4.99 flat	Co-located option once media > 10 GB — but R2's zero-egress is better for PHI signed URLs
AWS S3	$0.023/GB	$0.09/GB egress	~$1–5 + egress	Most expensive egress — skip

Use Cloudflare R2. Free 10 GB tier covers early PHI volume, **zero egress** means signed-URL downloads never generate a bandwidth bill, and it's S3-API compatible so `django-storages` "just works."

PHI hardening — this is medical data, configure it private:

```
# settings.py
STORAGES = {"default": {"BACKEND": "storages.backends.s3.S3Storage"}}
AWS_S3_ENDPOINT_URL = env("R2_ENDPOINT")          # https://<acct>.r2.cloudflarestorage.com
AWS_STORAGE_BUCKET_NAME = "clink-media"
AWS_DEFAULT_ACL = None                             # NOT public-read
AWS_QUERYSTRING_AUTH = True                        # force signed URLs
AWS_QUERYSTRING_EXPIRE = 300                       # 5-min expiry on PHI links
AWS_S3_FILE_OVERWRITE = False
```

- Bucket **must be private** (no public r2.dev access, no public listing) — PHI must never be a guessable URL.
- Every served file goes through a **signed URL with short expiry** so links can't be shared/leaked indefinitely.
- Keep R2 credentials in `.env` (`R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_ENDPOINT`) alongside your existing secret pattern.
- Note for compliance: there is no MENA-region R2; data lands in CF's network. Document this in your `compliance` app's data-processing record.

2.5 The migrations risk (fix before any second deploy)

Your `.gitignore` has **"ignore all migration files"** — this is the single most dangerous line in the repo for production. Why it breaks reproducible deploys:

- Your build runs `migrate`, but the migration files **don't exist in the deployed code**, so Django either applies *nothing* (schema drift between dev and prod) or, worse, a fresh environment auto-generates *different* migration files than your dev box → divergent schemas, broken `--fake` reconciliation, and a data migration (the Palestinian-cities seed) that may run twice or not at all.

- For a PHI app this is a data-integrity hazard, not just an annoyance.

Prod-safe practice: 1. **Commit all migrations.** Remove the ignore line; `git add --force <app>/migrations/*.py` for every app, keep only `__pycache__` ignored. 2. Treat migrations as code: review them in PRs, never edit applied ones. 3. **Run `migrate` in a release/pre-deploy phase, not in web boot.** On Render use a release command; on a VPS make `migrate` a step in your deploy script *before* restarting Gunicorn — never inside `wsgi.py` or worker startup (concurrent workers racing `migrate` causes lock contention and partial schema). Your current build string (`… collectstatic && migrate`) is acceptable on single-instance PaaS but **must move to a single-run release step before you scale to multiple web instances.**

2.6 Static files & CDN

- **WhiteNoise is correct at this scale — keep it.** With `CompressedManifestStaticFilesStorage` it serves hashed, immutable, far-future-cached assets directly from Gunicorn. No separate static host needed at MVP. Confirm `STORAGES["staticfiles"]` uses the manifest backend so filenames are content-hashed (enables `Cache-Control: immutable`).
- **Add Cloudflare (free plan) in front of the whole site** — not just for media. It gives you: free CDN edge caching of your WhiteNoise immutable assets (the hashed filenames mean CF can cache them forever safely), free TLS, DDoS protection, and **caches your render-blocking third-party assets** (Cairo/Inter Google Fonts, Font Awesome) closer to MENA users. Set Cloudflare SSL mode to **Full (strict)** so it composes with your origin Caddy/Render TLS and your `SECURE_SSL_REDIRECT` /HSTS without redirect loops.
- This composes cleanly: WhiteNoise sets `immutable` + hashed names → CF respects and edge-caches them → origin only serves cache misses. Don't "Cache Everything" on HTML (it's PHI-bearing, per-session) — only let CF cache `/static/*` .

2.7 Scaling path

Vertical first (cheapest, do this for a long time): - Bump the VPS: Hetzner CX22 → **CX32** (4 vCPU / 8 GB, ~€6.80/mo) or the ARM **CAX21** (4 vCPU / 8 GB, ~€6.49/mo — best value). Raise Gunicorn workers per the `(2×vCPU)+1` formula. One box serves a lot of clinics.

Horizontal (when one box maxes CPU/RAM): - You can run **multiple Gunicorn instances behind one load balancer** *because you already externalized state* — Redis cache/locks (shared), DB sessions (shared), DB (shared). **The one prerequisite you must finish first is §2.4 media on R2** — otherwise instance A can't serve a file instance B stored. Do the media fix and horizontal scaling is unlocked for free. - At that point add **managed Postgres + managed Redis** (Hetzner-adjacent, or DO managed PG ~$15/mo) so the DB isn't tied to one VPS, and put **PgBouncer** in front (§2.2).

Health checks & zero-downtime deploy: - Add a cheap `/healthz/` view returning 200 (checks DB + Redis reachability) for the LB/PaaS probe. Render/Railway use it natively; on a VPS, Caddy + a `systemctl reload` of a socket-activated Gunicorn gives near-zero-downtime restarts. Or run two Gunicorn `systemd`

units and reload them one at a time behind Caddy. - Deploy order, always: **migrate (release phase) → collectstatic → reload web → reload worker.**

Cheap staging: - A second small Hetzner box (CX22, or the cheaper ARM **CAX11** ~€3.79/mo) spun up as a clone (~€4–5/mo) with its own `.env` (`DEBUG=0` so your `manage.py check --deploy` and the `accounts.E001/E002` custom checks actually run pre-prod), a throwaway Postgres, and a separate R2 bucket. Tear it down between releases to save money, or use a Render free-tier service for PR previews.

2.8 Two-tier cost summary

Launch / MVP (cheapest viable) — ~$5–7/mo all-in: - Hetzner CX22 (Caddy + Gunicorn + Postgres + Redis on one box): **~€4.49 + €0.50 IPv4 ≈ $5/mo** (or 8 GB ARM CAX21 ≈ $7) - Cloudflare R2 media (under 10 GB free tier): **~$0** - Cloudflare CDN + DNS + TLS: **free** - Email Brevo / SMS TweetsMS: pay-per-use, separate - Background jobs: **sync at MVP, $0** - **Total compute+db+redis+media+CDN ≈ $5–7/mo** (fallback Hostinger VPS ≈ $7–9)

Growth / scale — ~$45–80/mo: - Hetzner CX32 / ARM CAX21 web (×1–2) **~€7–14** - Managed Postgres + PgBouncer **~$15–20** - Managed/standalone Redis **~$10** - RQ/Django-Q worker (2nd process/service) **~$0–7** - Cloudflare R2 media (50–100 GB, zero egress) **~$1–2** - Staging box **~$4–7** - **Total ≈ $45–80/mo**

> Pricing is 2026; VPS intro rates differ from renewal — **create the Hetzner account and confirm MENA payment acceptance before you build; Hostinger VPS is the ready fallback**.

Sources: - Hetzner Cloud pricing · Hetzner June 2026 price adjustment · Hetzner Object Storage - Hostinger VPS plans · Hostinger VPS pricing 2026 - Render pricing · Render pricing 2026 analysis - Fly.io pricing · Railway vs Fly.io 2026 - Cloudflare R2 pricing · Backblaze B2 pricing 2026

---

3. Security & Data Protection

Your app-level posture is genuinely strong (MFA/TOTP with Fernet-encrypted secrets, escalating brute-force lockouts, PHI export guards, hardened sessions, `check --deploy` gating). The gap is the **infra and supply-chain layer** plus a few latent app-config bugs. This section audits both. Costs are 2026 USD, monthly.

3.1 TLS/SSL and the `SECURE_PROXY_SSL_HEADER` trap

Your settings set `SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")` whenever `DEBUG=0` . **This is a security bug waiting to happen, not just a config choice.** If Gunicorn is ever reachable on a public port without a TLS-terminating proxy in front (a misconfigured firewall, a `0.0.0.0:8000` bind, a direct container port), an attacker sends `X-Forwarded-Proto: https` and Django believes the connection is secure — defeating `SECURE_SSL_REDIRECT` and serving Secure cookies over cleartext.

Mitigations (do all three): - **Bind Gunicorn to localhost only:** `gunicorn clinic_website.wsgi:application --bind 127.0.0.1:8000` so it is physically unreachable except via the proxy. On Render/PaaS this is handled for you; on a Hetzner/VPS it is your responsibility. - **Have the proxy strip and re-set the header.** With Caddy this is automatic; with Nginx, explicitly `proxy_set_header X-Forwarded-Proto $scheme;` so the client-supplied value can never pass through. - The header trust is only safe behind *one* trusted hop. If Cloudflare is also in front (recommended below), you have two hops — make sure the origin proxy is the one setting the header, and use Cloudflare's "Full (Strict)" SSL mode so Cloudflare→origin is also TLS.

Free certs — recommendation by tier:

Setup	TLS source	Cost	Why
Hetzner/VPS (RECOMMENDED for budget)	**Caddy** as the reverse proxy	$0	Caddy auto-provisions + auto-renews Let's Encrypt with zero config; a 3-line `Caddyfile` replaces an Nginx+certbot+cron stack. Sets `X-Forwarded-Proto` correctly out of the box.
VPS, Nginx preferred	certbot (`--nginx`)	$0	More moving parts (renewal cron/systemd timer); only pick if you already know Nginx.
Render / PaaS	Platform-managed TLS	$0	Automatic; nothing to wire.

Put **Cloudflare in front of either** (next section) and set SSL mode to **Full (Strict)** — never "Flexible" (Flexible = cleartext origin = PHI exposure).

HSTS preload is already on (1yr + `includeSubDomains` + `preload`). Do **not** submit to hstspreload.org until the final brand domain is chosen and you are certain every current and future subdomain (including any `api.` , `staff.`) can do HTTPS forever — preload is hard to reverse. Submit the apex of the *real* domain only, after go-live is stable for ~1 week. Keep the throwaway Hostinger `.com` **out** of preload.

3.2 WAF + DDoS + bot mitigation (Cloudflare)

Your Redis throttles protect *login/OTP/export* endpoints but do nothing against volumetric L7 floods, credential-stuffing across many IPs, or scrapers hitting public clinic/doctor pages. Put **Cloudflare** in front (DNS proxied, orange cloud). It composes cleanly with your Redis limits: Cloudflare drops junk at the edge before it reaches Gunicorn; your Redis limits remain the precise per-account/per-IP backstop for authenticated abuse.

Plan	Cost	What you get	Pick
Free	**$0**	Unmetered DDoS protection, basic Bot Fight Mode, 5 custom WAF rules, Full(Strict) TLS, caching	**RECOMMENDED at launch**
Pro	**$20/mo** (annual) / $25 monthly	20 custom WAF rules, **Super Bot Fight Mode**, managed WAF rulesets, 2 rate-limiting rules (1-min window)	Upgrade once you have real traffic/abuse
Business	$200/mo	100 rules, regex, 5 advanced rate-limit rules	Overkill until scale

Concrete free-tier setup for a healthcare app: - **Bot Fight Mode: ON.** Add a WAF custom rule to **challenge** (Managed Challenge) requests to `/accounts/login`, the OTP, and the JWT `/api/token/` endpoints — this stops most credential-stuffing before it burns your Redis lockout budget. - **Block by geography if your market is MENA-only:** a custom rule allowing only the countries you serve cuts the global bot floor dramatically. Use Managed Challenge rather than hard block initially to avoid locking out diaspora users. - **Cache static assets** (your WhiteNoise CSS, fonts) at the edge — security *and* the page-speed win. - **MENA/Palestine note:** Cloudflare's signup and Free/Pro plans are payable by international card and are not geo-restricted for Palestinian founders — this is one of the few enterprise-grade tools that "just works" from the region. Pro's $20 is the single highest-value security dollar you can spend after launch.

3.3 Secrets management

Today secrets live in `.env`-on-disk via `python-dotenv`. That is fine for a VPS *if* the file is `chmod 600`, owned by the app user, and **never** in a world-readable deploy dir — but it is the wrong default for a PaaS.

- **On Render/PaaS:** move every secret to the platform **environment-variable / secret store** (Render Environment Groups). Stop shipping a `.env` file; let `os.environ` win. Keep `.env.example` (already present) as the contract.
- **On the VPS:** keep `.env` but lock it down — `chown appuser:appuser .env && chmod 600 .env`, outside the git working tree, loaded by the systemd unit via `EnvironmentFile=`.
- `SECRET_KEY` **rotation:** rotating it invalidates all sessions (DB-backed) and any `signing`-based tokens — acceptable, plan a logout-everyone window. Keep it ≥50 chars, generated, unique per environment.
- ⚠️ `MFA_SECRET_KEY` **is special — do NOT rotate casually.** It is the Fernet key encrypting every stored TOTP secret at rest. Rotating it **bricks every staff member's authenticator** (their encrypted secrets become undecryptable) and forces a full re-enrolment + backup-code reset. If you must rotate (suspected key compromise), do it as a planned migration: decrypt-with-old → re-encrypt-with-new in a one-off script using Fernet **MultiFernet** (old+new keys) so you can roll forward without downtime, then drop the old key. Document this; it is the most dangerous lever in your codebase.
- **Pre-commit secret scanning — RECOMMENDED:** `gitleaks` ($0, MIT). Add a `.pre-commit-config.yaml` hook; it's <1s offline on a diff, catches AWS/DB/`SECRET_KEY` patterns before they're committed. Run **TruffleHog** *once* now against full git history (it verifies live credentials) since your repo predates scanning — then keep gitleaks at commit and in CI. Given your `.env` discipline this is cheap insurance, especially with PHI in scope.

3.4 Dependency & code security (all free)

You have `pandas` / `numpy` / `scikit-learn` / `cryptography` / `pillow` — a large attack surface and frequent CVEs. Wire these into GitHub (free, and you'll add CI anyway): - **Dependabot** (free on GitHub): enable version + **security** updates on `requirements.txt`. Pin Django to `~=6.0.x` and let Dependabot PR the patch bumps; Django security releases are frequent and you must stay current on 6.x. - `pip-audit`

in CI (`pip-audit -r requirements.txt`) — fails the build on known-vulnerable deps. Cheaper signal than Safety's paid tiers. - **Bandit** (`bandit -r . -x tests,migrations`) for Python SAST — catches hardcoded secrets, weak crypto, `subprocess` /SQL issues. - **CodeQL** (free for the repo on GitHub Actions) — deeper dataflow SAST, run weekly. - **`python manage.py check --deploy` in CI** — you already enforce custom checks `accounts.E001/E002` ; running the full `--deploy` in the pipeline catches any future regression in your security settings before deploy. This is your single best CI guardrail given how much security lives in `settings.py` .

3.5 Security headers beyond what's set

`X-Frame-Options` (via `XFrameOptionsMiddleware`) and `nosniff` are Django defaults and on. Add the rest:

- **Easy wins (set today in `settings.py`):**
- `SECURE_REFERRER_POLICY = "strict-origin-when-cross-origin"` (Django sets `same-origin` by default — fine, but pin it explicitly).
- `SECURE_CROSS_ORIGIN_OPENER_POLICY = "same-origin"` (Django default is already `same-origin` since 4.0 — confirm not overridden).
- **Permissions-Policy** and **CORP** are *not* set by core Django — add them via a tiny middleware or your proxy: `Permissions-Policy: camera=(), microphone=(), geolocation=(self), payment=()` and `Cross-Origin-Resource-Policy: same-origin` .
- **CSP — the hard one.** Django 6.0 ships **first-party CSP support** (`django.middleware.csp.ContentSecurityPolicyMiddleware` + `SECURE_CSP` / `csp_nonce`) — you don't even need the external `django-csp` package. **But your `base.html` templates load Google Fonts (Cairo/Inter) and Font Awesome from third-party CDNs via render-blocking `<link>` , and HTMX partials (`ws_notes` , `ws_orders` , `ws_prescriptions`) imply inline `hx-on:` /inline `<script>` / `<style>` blocks.** A naive `script-src 'self'` will break the app. Realistic path: 1. Start in **Report-Only** (`SECURE_CSP_REPORT_ONLY`) so nothing breaks while you collect violations. 2. Allowlist the CDNs explicitly: `style-src 'self' fonts.googleapis.com cdnjs.cloudflare.com; font-src 'self' fonts.gstatic.com; script-src 'self' 'nonce-{csp_nonce}'` . 3. **Self-host the fonts and Font Awesome** (drop them into WhiteNoise/static). This both shrinks the CSP to `'self'` *and* fixes the render-blocking third-party CDN performance problem — a two-birds change. Then CSP becomes trivial. 4. Add `{{ csp_nonce }}` to remaining inline `<script>` / `<style>` blocks. ⚠️ **HTMX caveat:** do **not** enable `htmx.config.inlineScriptNonce` — auto-injecting your nonce onto swapped-in inline scripts defeats the entire point of nonce-CSP. Keep inline JS out of HTMX response fragments instead.
- This is a "soon after launch" item, not a go-live blocker — but Report-Only CSP costs nothing to turn on now.

3.6 PHI / data protection

- **Encryption at rest:** use **managed Postgres** (Render Postgres, Supabase, Neon, or DigitalOcean) — all encrypt data + backups at rest by default. If self-hosting Postgres on the VPS (your launch setup), enable LUKS full-disk encryption; do **not** run unencrypted PHI on a bare VPS disk.
- **Least-privilege DB user:** the app must connect as a role that is **not** the Postgres superuser/owner — grant only `CONNECT`, `SELECT/INSERT/UPDATE/DELETE` on the app schema, no `CREATEDB` / `SUPERUSER`. Run migrations as a separate, more-privileged migration user in the deploy step only.
- **Backups:** automated daily managed backups + **test a restore** before launch (an untested backup is not a backup). Encrypt off-site copies. Retain per a written policy (e.g. 30 days operational + longer cold archive for medical-record retention norms).
- 🔴 **Media uploads are your biggest PHI hole.** `MEDIA_ROOT = BASE_DIR/media` on an ephemeral PaaS disk means **lab scans / prescription images are lost on redeploy** and unscannable cross-instance. Move to **object storage** (covered in the storage section) and serve PHI files via **short-lived signed/presigned URLs only** — never public-read buckets. Wire `django-storages` with `private` ACL; generate signed URLs scoped to the authenticated, clinic-isolated request.
- **Secure file-upload handling:** you already ship `python-magic-bin` — *use it*: validate true content-type by magic bytes (not the client `Content-Type` or extension), enforce a strict allowlist (PDF/JPEG/PNG/DICOM), and a hard size limit (`DATA_UPLOAD_MAX_MEMORY_SIZE` + per-field checks). Add **ClamAV** AV scanning on uploads — on a VPS run `clamd` and scan via `pyclamd` before persisting; on PaaS, scan in a worker. Malicious uploads in a clinic context are a real vector (a "lab result" PDF from a patient account).
- **Audit logging of PHI access (recommend — add if absent):** a `compliance` app exists; ensure it records *who viewed/exported which patient record, when, from where*. At minimum log read access to medical records and every CSV/bulk export (you already guard exports — log them too). Append-only, retained, and **excluded from normal deletion**. This is the single most-asked-for artifact in any healthcare incident review.
- **PII in logs / Sentry:** if you add error tracking (recommended), **scrub before send**. Sentry Developer plan is **$0** (5K events/mo) and enough at launch; Team is **$26/mo** (50K events). Set `send_default_pii=False`, add a `before_send` hook that strips request bodies, phone numbers, national IDs, and medical fields, and turn on Sentry's server-side data scrubbing. Never let a stack trace ship a patient's record into a third-party tracker. Apply the same scrubbing to your own application logs (no PHI in `INFO` / `DEBUG`).

3.7 CSRF/CORS and the JWT/DRF surface

- **JWT refresh rotation is currently OFF — fix it.** Your `settings.py` has `ROTATE_REFRESH_TOKENS: False` and `BLACKLIST_AFTER_ROTATION: False`, and `accounts/api_views.py` only blacklists on explicit logout *if the blacklist app is installed* (your TODO notes it may not be). For PHI, set: `python "ROTATE_REFRESH_TOKENS": True, "BLACKLIST_AFTER_ROTATION": True,` and add `rest_framework_simplejwt.token_blacklist` to `INSTALLED_APPS` (then migrate). Without the

blacklist app installed, a stolen 1-day refresh token cannot be revoked at all. With 60-min access / 1-day refresh, rotation + blacklist is the difference between "revocable" and "valid for 24h no matter what."

- **Token storage:** if the JWT API is consumed by your own web frontend, prefer **HttpOnly Secure cookies** over `localStorage` (XSS can read `localStorage`; your CSP work above is the other half of this defense). If it's a machine/partner API, document that tokens are bearer secrets.
- **CORS:** if you add `django-cors-headers`, set `CORS_ALLOWED_ORIGINS` to an explicit allowlist of *your* domains only — never `CORS_ALLOW_ALL_ORIGINS=True` on a PHI API. Keep `CORS_ALLOW_CREDENTIALS` aligned with cookie-based auth.
- **CSRF:** when the final domain is set, populate `CSRF_TRUSTED_ORIGINS` with the real `https://` brand domain (and any `staff.` / `api.` subdomains) — easy to forget at cutover and it breaks every POST.

3.8 Compliance framing (MENA / Palestine — practical, not legalistic)

There is no local HIPAA-equivalent statute binding you in Palestine, but you hold real medical records and your sub-processors (Brevo, Twilio, TweetsMS, hosting, object-storage) span jurisdictions. Align to **HIPAA/GDPR-style best practice** because it's the defensible baseline and your enterprise clinic customers will eventually ask: - **Publish a Privacy Policy + explicit patient consent** at signup (Arabic-first, given RTL/`ar` default), stating what data you hold and that you process medical data. - **Maintain a sub-processor list** (Brevo = email/PII, Twilio + TweetsMS = phone numbers/OTP, hosting = everything, object storage = medical images) and sign **DPAs** where offered (Brevo, Twilio, Sentry, Cloudflare all provide them; TweetsMS likely does not — note the residual risk that Palestinian phone numbers + OTP transit a local provider without a formal DPA). - **Data minimization:** don't send PHI to comms providers — SMS/email should carry "you have a new result, log in to view," never the result itself. - **Write a one-page breach-response plan**: who is notified, how you revoke tokens/sessions (you have the lockout + blacklist tooling once 3.7 is fixed), how you notify affected patients.

3.9 Prioritized checklist

🔴 **Fix before go-live (cheap, high-impact):** 1. Bind Gunicorn to `127.0.0.1` and verify the origin is unreachable except via the proxy (kills the `SECURE_PROXY_SSL_HEADER` spoof). 2. Cloudflare Free in front, SSL **Full (Strict)**, Bot Fight Mode on, challenge on login/OTP/`/api/token/`. 3. Move media uploads off the ephemeral disk to private object storage with signed URLs (PHI loss + exposure risk). 4. JWT: enable `ROTATE_REFRESH_TOKENS` + `BLACKLIST_AFTER_ROTATION`, install the blacklist app. 5. Managed Postgres (encrypted at rest + backups) and a least-privilege app DB role; **test one restore**. 6. Enforce `python-magic` content-type + size limits on every upload; allowlist file types. 7. Set `CSRF_TRUSTED_ORIGINS` / `CORS_ALLOWED_ORIGINS` to the real domain at cutover. 8. `gitleaks` pre-commit + one TruffleHog full-history scan; confirm no `.env` ever committed. 9. Privacy policy + consent (Arabic) and a sub-processor list live at launch.

🟡 **Soon after launch:** 10. CI pipeline running `check --deploy`, `pip-audit`, Bandit, Dependabot, CodeQL. 11. CSP in Report-Only → enforced; self-host Cairo/Inter/Font Awesome (security + perf). 12.

ClamAV scanning on uploads. 13. Sentry (free tier) with `send_default_pii=False` + PHI-scrubbing `before_send`. 14. PHI-access audit logging in the `compliance` app (who-viewed-what); append-only retention. 15. Permissions-Policy / CORP / explicit Referrer-Policy headers. 16. Document the `MFA_SECRET_KEY` MultiFernet rotation runbook; submit final domain to HSTS preload after a stable week. 17. Cloudflare Pro ($20/mo) once real traffic/abuse appears.

Approx. added monthly security spend: $0 at launch (Cloudflare Free, gitleaks, Dependabot/CodeQL/pip-audit/Bandit, Sentry Free, Let's Encrypt via Caddy all free) → **~$20–46/mo at growth** (Cloudflare Pro $20 + Sentry Team $26). Encryption-at-rest and managed backups ride on the managed-Postgres/object-storage line items in the infra section, not here.

Sources: Cloudflare Pro plan · Cloudflare pricing 2026 comparison · Cloudflare WAF rate limiting docs · Sentry pricing · Sentry PII & data scrubbing · Django 6.0 CSP docs · HTMX + CSP nonce caveat · Gitleaks vs TruffleHog 2026

---

4. Domain & DNS Management

You have one `.COM` at Hostinger that you're treating as throwaway/testing. The single highest-leverage move is to **put Cloudflare in front of it today** — not because the testing domain matters, but because doing it now means the eventual brand-domain cutover is a 30-minute, near-zero-downtime operation instead of a panicked DNS scramble. Cloudflare also gives you free WAF, CDN, and TLS that directly help a PHI app on a budget reverse proxy (it satisfies your `SECURE_PROXY_SSL_HEADER` TLS-termination assumption at the edge).

4.1 DNS host: Cloudflare (free) over Hostinger DNS — recommended now

Option	Monthly cost	Why / why not
Cloudflare Free DNS ✅ **RECOMMENDED**	**$0**	Fast anycast DNS, free Universal SSL, basic managed WAF, unmetered DDoS, CDN/caching for your WhiteNoise CSS + CDN fonts, analytics, CNAME flattening at apex, and full-zone export/import makes the brand cutover trivial. Pay with any international card — no MENA geo-restriction.
Hostinger DNS	$0 (bundled)	Works, but no edge WAF/CDN/proxy, weaker analytics, and you'd reconfigure everything from scratch at cutover.
Route 53	~$0.50/zone + queries	Overkill and adds an AWS billing relationship you don't need yet.

Cloudflare's Free plan is genuinely unlimited-time and includes Universal SSL + basic WAF + unmetered DDoS mitigation — sufficient for launch. Upgrade to **Pro ($20/mo, growth tier)** only when you want the full OWASP managed ruleset, image optimization, and better bot rules in front of PHI.

Exact steps to move the testing domain's nameservers to Cloudflare: 1. Cloudflare dashboard → **Add a site** → enter `your-testing.com` → choose **Free**. 2. Cloudflare auto-imports existing records — **review**

them, especially any A record pointing at your current host/Render. 3. Cloudflare shows two assigned nameservers, e.g. `xena.ns.cloudflare.com` / `rob.ns.cloudflare.com`. 4. Hostinger → **Domains** → **your domain** → **DNS / Nameservers** → **Change nameservers** → **Use custom nameservers** → paste both Cloudflare NS values → save. 5. Wait for propagation (minutes–24 h). Cloudflare emails you when the zone is **Active**. 6. Set **SSL/TLS mode = Full (strict)** (not Flexible — Flexible breaks your `SECURE_SSL_REDIRECT` into an infinite loop and sends plaintext PHI to your origin).

4.2 The full record set this app needs

App / web (apex + www): - `A` (or `AAAA`) at apex `@` → your host's IP, **Proxied (orange cloud)**. If your host gives a hostname not an IP (common on PaaS like Render), use a `CNAME @ → app.onrender.com` — Cloudflare's **CNAME flattening** makes an apex CNAME legal. - `CNAME www → @` (or the same host), Proxied. - Pick a canonical host (see 4.3) and 301 the other with a Cloudflare **Single Redirect / Bulk Redirect** rule (both available on Free; "Include subdomains" lets one rule cover apex+www).

Email deliverability for Brevo (transactional email — this is the part that silently breaks if skipped):

Purpose	Type	Host / Name	Value (from Brevo dashboard)
Brevo ownership	`TXT`	`@` (apex)	`brevo-code:xxxxxxxx` (your code)
DKIM	`TXT`	`brevo._domainkey`	`k=rsa; p=…` (Brevo gives the exact key)
DMARC	`TXT`	`_dmarc`	`v=DMARC1; p=none; rua=mailto:dmarc@yourbrand.com; fo=1`
SPF	`TXT`	`@`	only needed if you take a **dedicated IP** later: include `include:spf.brevo.com`; skip on shared sending

Notes specific to your stack: - **SPF/MX are NOT required for Brevo on shared sending** — Brevo only issues SPF/MX records when you provision a **dedicated IP**, which needs ~50k–100k emails/week to warm up. A solo clinic app will **not** hit that for a long time — stay on shared sending and skip SPF/MX. ✅ cheapest viable. - **DKIM is the one that matters** — add the `brevo._domainkey` TXT exactly as Brevo shows it, then click **Verify** in Brevo. Without it, appointment/OTP-fallback emails land in spam. - **DMARC: ramp it.** Start `p=none` (monitor only) → after ~2–4 weeks of clean DKIM-aligned reports, move to `p=quarantine` → then `p=reject`. Use a free aggregate-report inbox (e.g. `dmarc@` you own, or a free DMARC analyzer) for the `rua=`. - **TweetsMS** is SMS over HTTP API — **no DNS records needed**. Just keep its API key in `.env`. - **MX** only if you want to *receive* mail at the domain. For a launch, route `info@` / `support@` through a cheap mailbox (Zoho Mail free tier, or Brevo doesn't host mailboxes) and add its MX/SPF then — not required to *send*.

On `_domainkey` records and the Cloudflare proxy: TXT/DKIM/DMARC/MX records are **DNS-only (grey cloud)** by nature — they can't be proxied, and Cloudflare leaves them grey automatically. Only your A/CNAME web records get the orange cloud.

4.3 Subdomain strategy (clean scheme)

- **App = apex** `yourbrand.com`, with `www` 301-redirecting to apex. Rationale: shortest to type/SMS (you send appointment links via TweetsMS — every character counts in an SMS), and Cloudflare CNAME-flattening removes the historical apex limitation. Pick apex-canonical and stick to it everywhere (`ALLOWED_HOSTS`, canonical tags, PWA `start_url`).
- `staging.yourbrand.com` → your staging/preview deploy. **DNS-only or Proxied + Cloudflare Access** (free for up to 50 users) so PHI-shaped staging data isn't publicly crawlable.
- **Sending subdomain (optional, growth):** when you later authenticate Brevo on a **subdomain** like `mail.yourbrand.com` instead of the apex, a deliverability problem on marketing blasts can't damage the apex's reputation that serves OTP/appointment mail. For launch on shared IP, apex auth is fine; revisit at scale.
- Avoid scattering app functions across many subdomains — your `ClinicIsolationMiddleware` already does multi-tenant isolation in-path, so you do **not** need per-clinic subdomains (which would each need their own TLS/host config). Keep one host.

4.4 Testing → final brand-domain cutover (the important part)

Because the testing domain is throwaway, the goals are: (a) make the switch fast and low-downtime, (b) don't let the test domain pollute search results or burn an HSTS preload, (c) don't lose the brand domain to a typo/transfer-lock mistake.

Before you buy: protect the test domain from SEO + preload damage now - Serve `X-Robots-Tag: noindex, nofollow` (or a `robots.txt` `Disallow: /` + a `<meta name="robots" content="noindex">` in your base templates) on the **testing** domain so Google never indexes the throwaway brand. Easiest: a Cloudflare **Transform Rule** adding the `X-Robots-Tag` response header for that zone. - **Do NOT submit the testing domain to the HSTS preload list**, and do **not** set `SECURE_HSTS_PRELOAD=True` while that throwaway domain is live in prod — preload is effectively permanent and would force HTTPS on a domain you're about to abandon. Keep HSTS preload **off until the real brand domain is stable** (see step 9).

Cutover runbook (when the brand domain is purchased):

1. **Buy the brand** `.com` at a registrar with free WHOIS privacy + registrar-lock. Hostinger renewal is ~**$19.99/yr** with **free WHOIS privacy**; **Cloudflare Registrar** (~**$10–11/yr, at-cost, free WHOIS privacy, no markup**) is the better long-term home if you want registrar+DNS in one place — but it's transfer-in only (you can't register a brand-new name there until it's registered elsewhere first; buy at Hostinger/Porkbun, then transfer in later).
2. **Add the brand domain to Cloudflare** (same flow as 4.1). Because the test zone is already in Cloudflare, **export the test zone file** (DNS → Export) and **import** it into the new zone, then fix the A/CNAME targets to the same host. Replicating records is now copy-paste.
3. **Re-verify Brevo on the new domain:** add the new `brevo-code` TXT, the new `brevo._domainkey` DKIM, and `_dmarc` (start `p=none` again — DMARC reputation doesn't transfer), click Verify in Brevo.

4. **Update Django settings** (these are the lines that will 400/403 your users if missed): - `ALLOWED_HOSTS = ["yourbrand.com", "www.yourbrand.com"]` - `CSRF_TRUSTED_ORIGINS = ["https://yourbrand.com", "https://www.yourbrand.com"]` - any `SITE_URL` / canonical / OG `og:url` / hardcoded absolute links in templates and email bodies - **JWT issuer/audience** if your simple-JWT config sets `ISSUER` / `AUDIENCE` to the old host (and decide whether old refresh tokens must keep validating during the window — if you pinned the issuer, rotate gracefully).
5. **PWA assets** (once you ship the manifest from your PWA section): update `start_url`, `scope`, `id`, and any absolute icon URLs in `manifest.webmanifest` to the new origin, and the `theme-color` /canonical in base templates. A stale `start_url` makes installed PWAs open the dead domain.
6. **Email From addresses:** change Brevo sender + your Django `DEFAULT_FROM_EMAIL` / transactional `from` to `no-reply@yourbrand.com` (must be on the newly DKIM-verified domain or deliverability tanks).
7. **301 redirects, both directions:** - On the **old testing zone** in Cloudflare: a Single/Bulk Redirect `*your-testing.com/* → https://yourbrand.com/$1` (301, preserve path). Keep this live for a while so any shared test links resolve. - On the **new zone**: `www → apex` (or your chosen canonical) 301 with "Include subdomains".
8. **Sitemap & robots:** flip the brand domain's `robots.txt` to **allow** crawling, point `sitemap.xml` at the new host, and submit the new domain in Google Search Console (the test domain stays `noindex`).
9. **HSTS preload — now and only now:** once the brand domain serves HTTPS cleanly and you're confident, set `SECURE_HSTS_SECONDS=31536000`, `SECURE_HSTS_INCLUDE_SUBDOMAINS=True`, `SECURE_HSTS_PRELOAD=True`, deploy, then submit `yourbrand.com` to hstspreload.org. Pitfall: every subdomain (including `staging.`) must also serve valid HTTPS once `includeSubDomains` + preload is on — make sure staging has a cert (Cloudflare Universal SSL covers it) before submitting.
10. `manage.py check --deploy` must still pass on the new host config (your custom `accounts.E001/E002` checks) before you flip traffic.

Downtime: because the host (origin) doesn't move — only the domain pointing at it does — there is effectively **zero app downtime**. Users on the old test link get a 301 to the new brand domain.

4.5 TLS, WHOIS, locks, renewal

- **TLS:** Cloudflare **Universal SSL (free, auto-renewing)** at the edge for both domains; SSL/TLS mode **Full (strict)** so the Cloudflare↔origin hop is also encrypted (mandatory for PHI in transit). If your origin host already issues Let's Encrypt (Render does), Full (strict) validates against it. No manual cert renewal to forget.
- **WHOIS privacy: free** at both Hostinger and Cloudflare Registrar — keep it on for the brand domain (hides the founder's personal details).
- **Registrar lock + auto-renew:** enable **registrar/transfer lock** and **auto-renew** on the brand `.com` the day you buy it — losing a launched brand domain to an expiry lapse or unauthorized transfer is an unrecoverable disaster. Add a calendar reminder ~30 days before expiry as a backstop.

- **Cost summary:** brand `.com` **~$10–20/yr** (Cloudflare Registrar ~$10–11 cheapest long-term; Hostinger ~$19.99 renewal) + **Cloudflare DNS/WAF/TLS $0/mo** at launch, optional **Pro $20/mo** at scale. Total domain+DNS spend at launch: **~$1–2/month amortized.**

4.6 Don't throw the test domain away — make it permanent staging

After the brand launches, **keep the test** `.com` as your permanent `staging` /preview environment instead of paying for another domain: - Point `staging.yourbrand.com` **or** the old test domain itself at your staging deploy. - Keep it `noindex` (already set above) and gate it with **Cloudflare Access (free ≤50 users)** so PHI-shaped seed data and pre-release features aren't publicly reachable. - Use it as the target for your future CI/CD (your repo currently has no `.github/workflows`) — deploy PRs there before promoting to the brand domain. This turns a ~$15/yr "wasted" renewal into your staging tier at no extra cost.

Sources: Cloudflare Free plan, Cloudflare Universal SSL docs, Cloudflare Redirects docs, Brevo domain authentication (DKIM/DMARC), Brevo dedicated IP requirements, Hostinger domain pricing 2026

---

5. CI/CD & Monitoring

This stack has zero CI/CD and no Dockerfile today, and the single most dangerous repo risk lives here: **all migrations are** `.gitignore` **d** (`# ignore all migration files`). That makes production `migrate` non-reproducible and silently divergent from dev. The pipeline below is built to (a) make that impossible to reintroduce, and (b) give a solo founder real observability for **~$0/month at launch**.

5.0 Prerequisite — fix the migration gitignore (do this first)

Nothing in CD is trustworthy until migrations are version-controlled.

```
# In .gitignore: DELETE the blanket "*/migrations/*.py" / "ignore all migration files" rule.
# Keep only:
#   */migrations/__pycache__/
# Then commit the real migration history:
git add -f */migrations/*.py
python manage.py makemigrations          # confirm tree is clean
git commit -m "chore: track migrations (required for reproducible deploys)"
```

The CI step `makemigrations --check --dry-run` (below) then **fails the build** if anyone adds a model change without a migration — turning the old footgun into a guardrail. Your Palestinian-cities seed data migration must be tracked too, or fresh prod DBs come up empty.

5.1 CI — GitHub Actions (RECOMMENDED)

GitHub Actions free tier = **2,000 Linux minutes/month** on private repos (the Jan 2026 repricing left the free allotment intact; overage is ~$0.006/2-core min). A pipeline like this runs in 2–4 min, so a solo

founder will essentially never pay. No Node/npm in the repo means CI stays lean (pure Python).

`.github/workflows/ci.yml` (outline):

```yaml
name: ci
on:
  pull_request:
  push: { branches: [main] }

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: clink }
        ports: ['5432:5432']
        options: >-
          --health-cmd "pg_isready" --health-interval 5s --health-retries 10
      redis:
        image: redis:7
        ports: ['6379:6379']
        options: --health-cmd "redis-cli ping" --health-interval 5s --health-retries 10
    env:
      DATABASE_URL: postgres://postgres:postgres@localhost:5432/clink
      REDIS_URL: redis://localhost:6379/0
      DEBUG: "0"                          # exercise the prod-hardening path
      SECRET_KEY: ci-dummy-not-secret
      MFA_SECRET_KEY: ${{ secrets.CI_FERNET_KEY }}   # valid Fernet key for tests
      # set the accounts.E001/E002 feature flags ON so check --deploy passes
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.13', cache: 'pip' }   # pip cache = fast installs
      - run: pip install -r requirements.txt
      - run: pip install ruff bandit pip-audit coverage

      - name: Lint
        run: ruff check .
      - name: Migrations are committed & complete   # CRITICAL guard
        run: python manage.py makemigrations --check --dry-run
      - name: Deploy checks (accounts.E001/E002, security)
        run: python manage.py check --deploy --fail-level WARNING
      - name: Tests + coverage
        run: |
          coverage run manage.py test
          coverage report --fail-under=50      # ratchet up over time
      - name: collectstatic smoke test (WhiteNoise manifest)
        run: python manage.py collectstatic --noinput
      - name: Security audit
        run: |
          pip-audit -r requirements.txt --strict
          bandit -r . -x ./*/migrations,./*/tests -ll
```

Notes specific to this repo: - Run with `DEBUG=0` so `check --deploy` and the custom `accounts.E001/E002` checks actually fire in CI (they're no-ops under DEBUG). - `collectstatic` is a real smoke test here because WhiteNoise uses a hashed manifest — a missing referenced static file breaks prod boot, not just the page. - `pip-audit` matters: you ship `cryptography`/Fernet, `psycopg2-binary`, `pillow`, `twilio` — all recurrent CVE sources for a PHI app. - Pin `psycopg2-binary` → consider `psycopg[binary]` later, but keep CI's Postgres service at the **same major (16)** as production.

Lighthouse CI (cheap perf budget): add a second optional job that boots Gunicorn against the CI Postgres and runs `lhci autorun` against the patient dashboard with a budget (e.g. LCP < 2.5s, render-blocking CSS budget). This directly catches your known issue: render-blocking Google Fonts (Cairo/Inter) + Font Awesome CDN `<link>`s in `<head>`. Gate as a **warning**, not a hard fail, at launch.

5.2 The `/healthz` endpoint you don't have yet (add it)

Confirmed: the repo has **no** health route. The load balancer, uptime monitor, and CD smoke check all need one. Add a tiny view that checks **DB + Redis** (both are your hard dependencies):

```python
# accounts/health.py  (wire at path("healthz/", healthz) — NO auth, NO tenant middleware)
from django.db import connection
from django.core.cache import cache
from django.http import JsonResponse

def healthz(request):
    try:
        connection.cursor().execute("SELECT 1")
        cache.set("healthz", "1", 5); assert cache.get("healthz") == "1"
    except Exception as e:
        return JsonResponse({"status": "fail", "error": str(e)[:120]}, status=503)
    return JsonResponse({"status": "ok"})
```

Important for this app: exempt `/healthz` from `ClinicIsolationMiddleware`, `SECURE_SSL_REDIRECT` loops, and login — and have it return `200` only when **both** Postgres and Redis answer (Redis backs OTP throttle + brute-force lockouts + MFA rate-limits, so a Redis outage silently breaks auth, not just cache). Optionally split `/readyz` (deep check, for the LB) from `/healthz` (shallow, for cheap uptime pings) to avoid hammering the DB every 60s.

5.3 CD — two paths

Run **`migrate` and `collectstatic` in a release/pre-deploy phase, never in Gunicorn boot.** Booting multiple Gunicorn workers that each race `migrate` is a classic PaaS data-corruption bug.

Option	Best when	Monthly cost	Rollback
(a) PaaS auto-deploy from GitHub (Render/Railway) ✅ RECOMMENDED for launch	Solo founder, want zero infra ops	$0 CI + PaaS plan from §hosting	One-click "Rollback to previous deploy" in dashboard
(b) VPS via SSH + docker-compose	You're on a Hetzner/Hostinger VPS; want lower steady-state cost	$0 CI + VPS cost	`git checkout <sha> && compose up -d` + restore DB snapshot

(a) PaaS (Render-style) — release command: Your `DEPLOY_RENDER.md` currently bundles migrate into *build*. Move it to the **Release phase** instead (Render "Pre-Deploy Command" / Railway release):

```
# Build:   pip install -r requirements.txt && python manage.py collectstatic --noinput
# Release: python manage.py migrate --noinput          # runs once, pre-traffic
# Start:   gunicorn clinic_website.wsgi:application
```

Trigger on merge to `main`. Either enable the platform's native GitHub auto-deploy, or call a **deploy hook** from a final CI job so deploy only fires after green tests:

```yaml
deploy:
  needs: test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - run: curl -fsS "$RENDER_DEPLOY_HOOK"     # secret; gated behind passing tests
```

(b) VPS — release + smoke + rollback (GitHub Actions `appleboy/ssh-action`):

```
cd /srv/clink && git fetch && git checkout "$GITHUB_SHA"
pg_dump "$DATABASE_URL" | gzip > /backups/pre-deploy-$(date +%F-%H%M).sql.gz   # backup BEFORE migrate
docker compose run --rm web python manage.py migrate --noinput
docker compose run --rm web python manage.py collectstatic --noinput
docker compose up -d --no-deps web
sleep 5 && curl -fsS https://DOMAIN/healthz || { docker compose restart web; exit 1; }  # smoke + auto-rollback signal
```

Optional Dockerfile (recommended for path (b), nice-to-have for (a)): a single `python:3.13-slim` image pins your runtime, system libs (`libmagic1` for `python-magic-bin` , `libpq5` , fonts), and Gunicorn flags in one reproducible artifact — and lets CI and prod run the *identical* environment. Worth it the moment you touch a VPS; skip it while on pure PaaS to stay simple.

Zero-downtime migration discipline (PHI = no maintenance windows you can hide): 1. Backward-compatible migrations only: add nullable columns first, backfill in a separate migration, *then* enforce `NOT NULL` — never rename/drop in the same deploy that ships code using the old name. 2. Split additive schema (deploy N) from destructive cleanup (deploy N+2), so an old Gunicorn worker mid-rollout never hits a missing column. 3. **Always `pg_dump` before `migrate`** (shown above). On managed Postgres, also rely on daily automated backups + PITR.

5.4 Monitoring & observability (Launch stack ≈ $0/mo)

Concern	RECOMMENDED tool	Free tier (2026)	When to pay
Errors (server + JS)	**Sentry Developer** ✅	Free forever, **5k events/mo**, 1 user	Team ~$26/mo when >5k events
Uptime / availability	**Better Stack Uptime** ✅	Free: **10 monitors, 3-min checks, status page, email/Slack**	$29/mo for 30-sec checks
RUM / Core Web Vitals	**Cloudflare Web Analytics** ✅	**Free, cookieless, no banner** (key for MENA/RTL privacy + no consent UI to build)	—

Concern	RECOMMENDED tool	Free tier (2026)	When to pay
Logs	Platform stdout logs → **Better Stack Logs**	3 GB / 3-day retention free	$24/mo for 30-day/10 GB
Lighthouse perf budgets	LHCI in GitHub Actions	Free (runs on your minutes)	—

Why these picks for this stack: - **Sentry** — install `sentry-sdk[django]`; it auto-instruments Django, Gunicorn, Redis, and the DB. **PHI scrubbing is mandatory, not optional**: set `send_default_pii=False`, and add `before_send` / `before_send_transaction` to strip patient identifiers, prescriptions, allergy data, and tokens out of request bodies, query params, and breadcrumbs before they leave your server. Also enable Sentry's server-side data-scrubbing rules as defense-in-depth. For the HTMX partials (`ws_notes`, `ws_orders`, `ws_prescriptions`) the browser SDK will capture URLs that may contain record IDs — scrub those too. Self-hosted **GlitchTip** is the alt if you ever decide PHI must never touch a US SaaS (relevant for some healthcare data-residency stances), but it adds ops you don't want at launch. - **UptimeRobot caveat:** its generous free 50-monitor plan is **non-commercial only since Oct 2024**, and Telegram alerting is paid-tier (Solo $9/mo). For a commercial clinic app, **Better Stack's free tier is the cleaner fit** (commercial-OK, includes a status page + Slack/email). Point either at `https://DOMAIN/healthz`. For free chat alerts, Cloudflare/Better Stack → email works day one; a Telegram bot webhook is the cheapest pager. - **Cloudflare Web Analytics** is the standout value here: free RUM + Core Web Vitals with **no cookie consent banner** — meaningful when your market is MENA/Palestine and you'd otherwise owe a consent UI in Arabic-RTL. It also confirms whether the render-blocking CDN fonts hurt real-user LCP.

Structured logging → stdout (works for both PaaS and a containerized VPS). Configure Django `LOGGING` to emit JSON to stdout so the platform/Better Stack ingests it; **never log PHI or auth secrets**. Alert on: - 5xx rate spike, any `/healthz` 503, repeated `migrate` failures. - Auth-security signals you already generate: brute-force lockout escalations, MFA failures, **bulk PHI CSV export** events (you have guards — log + alert on every bulk export as an audit/exfil tripwire).

DB + Redis health: enable Postgres `log_min_duration_statement = 500ms` (slow-query log) — your `pandas` / `scikit-learn` doctor-analytics paths are the likely offenders. Set a **Redis memory alert** (e.g. >75% maxmemory): if Redis evicts keys, OTP throttles and login lockouts degrade silently, which is a security event, not just a perf blip.

5.5 On-call & SLO (solo founder)

- **SLO:** target **99.5% monthly uptime** for a small clinic app (~3.6 h/mo error budget) — honest for a one-person team, still credible to clinics. Tighten to 99.9% only once you're on multi-instance hosting.
- **What pages you (Telegram/email, in priority order):** 1. `/healthz` down ≥ 2 consecutive checks (site or DB/Redis hard-down). 2. Sentry **new-issue** or error-rate spike alert. 3. CD deploy failed / post-deploy smoke check failed. 4. Bulk PHI export anomaly or repeated MFA/brute-force lockouts (security).

- Everything else (slow queries, Redis memory, coverage drift, Lighthouse regressions) is **dashboard-review, not a page** — batch it into a weekly look so you're not desensitized to alerts.

Growth / scale additions (defer until paying customers): Sentry Team ($26/mo) for >5k events + performance tracing, Better Stack paid ($29–53/mo) for 30-sec checks and 30-day log retention, a staging environment in CI before `main`, and self-hosted GlitchTip if a clinic contract demands PHI never leaves your infrastructure.

Sources: Sentry pricing · GitHub Actions 2026 pricing · Better Stack pricing · UptimeRobot pricing · Cloudflare Web Analytics docs

---

6. Additional Recommendations (Things Not To Forget)

These are the production-critical gaps the five sections above don't fully close. Everything below is cheap-first and tailored to a solo founder running Django 6 + Gunicorn + Postgres + Redis + WhiteNoise serving PHI to a MENA/Arabic-RTL market. **The single biggest landmines in this repo are operational, not architectural: migrations are `.gitignore`d and media is on local disk — both must be fixed before any of the backup/DR advice below is even meaningful.**

6.1 Fix the two repo blockers first (prerequisite, $0)

Nothing below works until these are done:

- **Un-ignore migrations.** Remove `ignore all migration files` from `.gitignore`, run `python manage.py makemigrations` for every app (you noted a Palestinian-cities seed migration + `0014_order_allergy_acknowledged_at` already untracked), and commit them. Without committed migrations, two deploys can produce two different schemas and a restore can't rebuild the DB. This is the highest-priority item in the whole plan.
- **Move media off local disk to object storage** (`django-storages[s3]` + Cloudflare R2). `MEDIA_ROOT = BASE_DIR/media` is wiped on every redeploy and breaks horizontal scaling. R2 is the right pick (see 6.2/6.9). Config: `DEFAULT_FILE_STORAGE` (or Django 5+ `STORAGES["default"]`) → `storages.backends.s3.S3Storage`, `AWS_S3_ENDPOINT_URL=https://<acct>.r2.cloudflarestorage.com`, keep `STATICFILES` on WhiteNoise (don't move static — WhiteNoise is fine and free).

6.2 Backups & Disaster Recovery

For a clinic holding prescriptions/allergies, **target RPO ≤ 15 min, RTO ≤ 2–4 hrs.** A backup you've never restored is not a backup.

- **Postgres: use managed PITR, don't hand-roll.** A managed Postgres with point-in-time-recovery is worth the few extra dollars vs `pg_dump` cron — it gives continuous WAL-based recovery (RPO minutes) with zero code. Most managed Postgres tiers include 7-day PITR.

- **Belt-and-suspenders nightly dump to R2** regardless: `pg_dump -Fc | gpg --encrypt` to an R2 bucket via a scheduled job (see 6.3). **Encrypt before upload** (PHI must never sit unencrypted in object storage). Retention: 7 daily + 4 weekly + 6 monthly is plenty early.
- **Media (R2): turn on Object Versioning / lifecycle** so an accidental delete or overwrite is recoverable; expire old versions after 30–60 days to control cost.
- **RESTORE DRILL — calendar it monthly.** Spin up your staging DB (6.5) from last night's encrypted dump, run `manage.py check` + `migrate --check`, log in. This doubles as proof for the trust pages (6.6).
- **Testing-stage data:** the throwaway Hostinger `.com` data is non-production — do NOT keep PHI-shaped real patient data there. Seed it with synthetic/fake records only.

Backup option	RPO	~Monthly	Pick
Managed Postgres PITR (Render/Railway/Neon tier)	~minutes	included in DB cost	**RECOMMENDED** — zero-maintenance
`pg_dump` cron → R2 (encrypted)	~24 hrs	~$0 (within R2 free 10 GB)	Add as secondary
Manual dumps only	days	$0	Not acceptable for PHI

6.3 Background jobs / async (you have Redis already — use it)

Right now SMS/OTP/email almost certainly fire **inside the request**, so a slow TweetsMS/Twilio/Brevo call blocks the user and a failed send is silently lost. Move all transactional sends to a queue with retries.

- **RECOMMENDED: Django-Q2** (not Celery). It's the best price/value here: simpler than Celery, reuses your **existing Redis** broker, has a built-in scheduler (no separate Celery-beat), and runs in one extra cheap worker process (`qcluster`). Celery is overkill for a solo founder; RQ lacks a built-in scheduler.
- Wrap each send in a task with `retry` / `max_attempts` and exponential backoff; log failures to Sentry.
- **Appointment reminders** = scheduled `Schedule` objects (e.g. hourly sweep for appointments 24h/2h out). This is the feature that most justifies the queue.
- Cost: one small worker dyno/process, ~$7/mo on most PaaS (or $0 if co-located on a VPS).

6.4 Email & SMS deliverability + cost

- **Brevo (email):** free tier is **300 emails/day, shared marketing+transactional** (source). Fine for launch; reset is daily, no rollover. When reminders + OTP-by-email push past 300/day, the paid tier starts ~$9/mo. **Configure SPF, DKIM, and DMARC** for the final domain (ties to the Domain/DNS section) or your OTP emails land in spam.
- **SMS:** Twilio Verify is **~$0.05/verification** plus per-SMS carrier fees; Palestine/Israel SMS is comparatively pricey (Twilio PS pricing). **Keep TweetsMS as primary** for cost in-region, **Twilio**

Verify as fallback — you already have both. Add automatic fallback: if TweetsMS API returns non-success, retry once then fail over to Twilio.

- **Reminder volume math:** 5 doctors × ~20 appts/day × 1 reminder ≈ 100 SMS/day ≈ 3,000/mo. At even $0.02–0.05/SMS that's ~$60–150/mo — **SMS will likely be your #1 variable cost**, so let patients opt into email/WhatsApp reminders to shave it.
- **Monitor bounces/failures:** Brevo webhook → log + Sentry alert on hard bounces; alert if SMS failure rate spikes.

6.5 Staging / pre-prod (reuse the throwaway Hostinger domain)

- Promote the existing test `.com` to a **permanent staging environment**: its own DB, its own R2 bucket, **synthetic data only**, `DEBUG=0` so you test the real production hardening (HSTS, SSL redirect, `check --deploy`).
- Keep it cheap: smallest DB tier + **scale-to-zero web service** (Render free/Railway sleep) so it costs ~$0 idle.
- `X-Robots-Tag: noindex` **on the entire staging host** + HTTP basic-auth in front of it so Google never indexes it and no real users wander in.

6.6 Legal / trust pages (PHI raises the bar)

- Ship **Privacy Policy, Terms of Service, and a cookie/consent notice** before launch. If you adopt cookieless analytics (6.8), you can avoid a heavy GDPR-style consent banner entirely.
- Maintain a **documented sub-processor list**: hosting, R2, Brevo, Twilio, TweetsMS, Sentry, Cloudflare — patients/clinics will ask where PHI flows.
- Add a **data-subject-request path** (export/delete-my-data) — you already have PHI CSV export guards; reuse that plumbing.
- Cheap route: a generator (Termly/iubenda free tier, ~$0–10/mo) for first drafts, then a one-time local-lawyer review. Provide **Arabic + English** versions (your market expects Arabic).

6.7 SEO & discoverability (public pages only — never PHI)

- `django.contrib.sitemaps` for `sitemap.xml` + a `robots.txt` (static via WhiteNoise).
- **Critical guardrail:** every authenticated/PHI view must emit `X-Robots-Tag: noindex, nofollow` (a small middleware keyed off `request.user.is_authenticated`), and the **staging host fully noindex** (6.5). Only the landing + public doctor-browse pages get indexed.
- `hreflang` for `ar` / `en` + a `canonical` tag (you already serve both locales); add Open Graph/Twitter cards and **schema.org** `MedicalClinic` / `Physician` **JSON-LD** on public doctor pages for rich results.

6.8 Analytics — cookieless to dodge the consent banner

For a health app, avoid GA4 (sets cookies → forces a consent banner over PHI pages).

Option	Cookies	~Monthly	Pick
Cloudflare Web Analytics	none	**$0**	**RECOMMENDED for launch** — free, 1-line script, no banner (details)
Plausible / Umami self-host	none	~$0 on existing VPS (needs ~2 GB RAM, Docker)	Growth — full retention, no 30-day/10%-sample limits
GA4	yes	$0	Avoid (consent + privacy optics)

Cloudflare's free tier samples to ~10% and keeps 30 days (source) — fine to start; move to self-hosted Plausible/Umami when you want real retention.

6.9 Cost-optimization tactics

- **Put Cloudflare in front of everything (free plan).** Caches static, terminates TLS (works with your existing `SECURE_PROXY_SSL_HEADER`), free WAF, and cuts origin egress.
- **R2 over S3 for media + backups: zero egress fees**, $0.015/GB storage, **10 GB free** (R2 pricing). Patient-uploaded images served via R2 cost effectively nothing in bandwidth — a real win vs S3 egress.
- **Sentry free Developer plan** = 5,000 errors/mo, 1 user, 30-day retention (source) — enough for launch; Team is $26/mo only when you outgrow it.
- **Bill annually** on your one or two committed services (hosting/DB) once stable — usually ~2 months free vs monthly.
- **Scale-to-zero staging** (6.5) and **right-size**: one small web + one small worker + smallest managed Postgres is a viable launch footprint.
- **Biggest cost drivers, in order:** (1) **SMS** (variable, MENA rates — cap it with email/WhatsApp opt-in), (2) managed Postgres, (3) web/worker compute. R2, analytics, and Sentry are near-zero early.

6.10 Data residency (PHI/MENA) & dependency cadence

- **Residency:** there is no in-Palestine cloud region; realistic PHI-friendly regions are **EU (Frankfurt/Amsterdam)** — good privacy regime, low-ish latency to MENA, and avoids US-jurisdiction concerns. Pick **one region** and keep DB, media (R2 location hint), and backups all in it; state it on your privacy page. Encryption at rest (you already use Fernet for MFA secrets) + R2/DB encryption covers the rest.
- **Founder payment/geo note:** Cloudflare, Sentry, Brevo, Twilio all accept international cards and serve MENA founders fine. Confirm your card works at signup for any **PaaS host** — some require a card a Palestine-based founder may find easier via Stripe-backed checkout (verify at signup).
- **Dependency-update cadence:** monthly `pip list --outdated`; subscribe to GitHub Dependabot/security alerts (free) on the repo; patch Django security releases within days (you're on 6.0.1 — track the 6.0.x line). Pin versions in `requirements.txt` and bump deliberately. Add a quarterly review of `cryptography`, `psycopg2-binary`, `simplejwt`, and `pillow` (the highest-CVE-surface deps here).

Sources: Brevo free-plan limits, Cloudflare R2 pricing, Sentry pricing, Twilio SMS Palestine pricing, Cloudflare Web Analytics vs Plausible

---

7. Consolidated Cost, Roadmap & Go-Live

Consolidated Monthly Cost

> All figures 2026 USD, approximate. VPS intro rates differ from renewal — verify at signup. "Free" = within a genuine free tier at launch volume.

Tier 1 — Launch / MVP (single VPS, lean)

Line item	$/mo	Notes
Hetzner CX22, Frankfurt (Caddy+Gunicorn+Postgres+Redis)	~$5	€4.49 + €0.50 IPv4. Fallback: Hostinger VPS ~$7–9
Cloudflare R2 media (< 10 GB PHI)	$0 (free)	Zero egress on signed-URL downloads
Cloudflare CDN + DNS + TLS + WAF Free	$0 (free)	Bot Fight Mode + challenge on auth endpoints
Brevo email	$0 (free)	300/day shared; DKIM/DMARC = $0
GitHub Actions CI	$0 (free)	~2–4 min/run vs 2,000 free min
Sentry Developer	$0 (free)	5k events/mo, PHI-scrubbed
Better Stack Uptime	$0 (free)	10 monitors + status page
Cloudflare Web Analytics	$0 (free)	Cookieless RUM/CWV
Brand `.com` domain (amortized)	~$1	~$10–20/yr (CF Registrar cheapest)
SMS (variable, pay-per-use)	**~$60–150**	~3,000 reminders/mo at MENA rates; **biggest cost**
TOTAL (fixed infra)	**~$6–8/mo**	Excludes SMS
TOTAL (with typical SMS)	**~$66–158/mo**	SMS dominates — cap with email/WhatsApp opt-in

Tier 2 — Growth / Scale

Line item	$/mo	Notes
Web compute (Hetzner CX32 / ARM CAX21 ×1–2)	~$7–15	Vertical first; horizontal once media on R2
Managed Postgres + PITR + PgBouncer	~$15–20	Encrypted backups + point-in-time recovery
Managed/standalone Redis	~$10	Decouple from web box

Line item	$/mo	Notes
Async worker (Django-Q2 `qcluster`, 2nd process/service)	~$0–7	$0 if co-located on VPS
Cloudflare R2 media (50–100 GB)	~$1–2	Still zero egress
Cloudflare Pro (WAF managed rules, Super Bot Fight)	~$20	Highest-value security dollar post-launch
Sentry Team	~$26	50k events + tracing, when >5k/mo
Better Stack paid (30-sec checks, 30-day logs)	~$0–29	Optional
Staging box	~$4–7	Or reuse throwaway domain on scale-to-zero
TOTAL (fixed infra)	**~$55–110/mo**	Plus SMS, still the variable driver

Biggest cost levers & money-saving tactics: - **SMS is your #1 variable cost** — MENA per-SMS rates make reminders ~$60–150/mo at modest volume. Offer email/WhatsApp reminder opt-in and keep TweetsMS primary (Twilio only as fallback) to slash it. - **Stay on one VPS as long as possible** (vertical scaling) and lean on free tiers (Cloudflare, R2, Sentry, Better Stack, Brevo, GitHub Actions) — fixed infra realistically stays under ~$12/mo at launch. - **Bill annually** on the one or two committed services (VPS/managed DB) once stable for ~2 months free, and use scale-to-zero staging so pre-prod costs ~$0 idle.

Phased Production Roadmap

Phase 0 — Pre-launch hardening (do these first; all ~$0, they gate everything else) - Remove the blanket migration ignore (`.gitignore` line 18 `# Migrations (ignore all migration files)`); `git add -f */migrations/*.py` , run `makemigrations` to confirm a clean tree, commit (incl. the Palestinian-cities seed and `0014_order_allergy_acknowledged_at` already untracked). - Move media to private Cloudflare R2 via `django-storages[s3]` : `AWS_DEFAULT_ACL=None` , `AWS_QUERYSTRING_AUTH=True` , `AWS_QUERYSTRING_EXPIRE=300` , `AWS_S3_FILE_OVERWRITE=False` . Keep static on WhiteNoise. - Add `/healthz` (DB + Redis check, 503 on failure) and `/readyz` ; exempt from `ClinicIsolationMiddleware` , auth, and SSL-redirect loops. - Bind Gunicorn to `127.0.0.1` (kills the `SECURE_PROXY_SSL_HEADER` spoof at settings.py:324); proxy strips/re-sets `X-Forwarded-Proto` . - JWT: set `ROTATE_REFRESH_TOKENS=True` + `BLACKLIST_AFTER_ROTATION=True` (currently `False` at settings.py:280-281) and install `rest_framework_simplejwt.token_blacklist` , then migrate. - Enforce `python-magic` true-content-type + size allowlist on every upload (PDF/JPEG/PNG/DICOM). - Stand up GitHub Actions CI: `makemigrations --check --dry-run` , `check --deploy` with `DEBUG=0` , tests+coverage, `collectstatic` smoke, `pip-audit` , Bandit, ruff. Move `migrate` to a release/pre-deploy phase (out of build/boot). - Secrets: `chmod 600 .env` outside the git tree (VPS) or platform secret store (PaaS); add gitleaks pre-commit + one TruffleHog full-history scan.

Phase 1 — MVP go-live (Launch tier, ~$8–12/mo fixed) - Provision Hetzner CX22 (Frankfurt) — create the account & payment first to confirm acceptance; add a 2–4 GB swap file; Caddy (auto Let's Encrypt) →

Gunicorn `--workers 5 --max-requests 1000` on a unix socket; local Postgres 16 (LUKS, least-priv app role) + Redis 7. - Put Cloudflare Free in front of the existing test `.com` : SSL **Full (strict)**, Bot Fight Mode on, Managed Challenge on `/accounts/login` , OTP, `/api/token/` ; cache `/static/*` only (never PHI HTML). - Brevo DKIM + DMARC (`p=none`) on the active domain; keep SPF/MX off (shared sending). - PWA basics ($0): self-host Cairo/Inter (arabic+latin subset, preload, `font-display:swap`) and drop the Google Fonts CDN; `CompressedManifestStaticFilesStorage` + `whitenoise[brotli]` ; `manifest.webmanifest` (`dir:rtl` , `lang:ar`) + maskable/apple-touch icons + theme-color; vanilla `/sw.js` at site root with PHI guards (skip `/media/` , `HX-Request` , `Authorization` , non-GET) + `/offline/` . - Monitoring: Sentry Developer (`send_default_pii=False` + `before_send` PHI scrub), Better Stack Uptime → `/healthz` , Cloudflare Web Analytics. - Legal: Arabic+English Privacy Policy + consent + sub-processor list live; nightly `pg_dump | gpg` → R2 + R2 object versioning; **run one restore drill**.

Phase 2 — Scale, brand cutover & resilience (Growth tier, ~$55–110/mo fixed) - Brand-domain cutover: buy `.com` (registrar lock + auto-renew + WHOIS privacy), import zone into Cloudflare, re-verify Brevo (new DKIM/DMARC), update `ALLOWED_HOSTS` / `CSRF_TRUSTED_ORIGINS` / `SITE_URL` /manifest `start_url` , 301 old→new both directions, then submit apex to HSTS preload after a stable week. Repurpose the old test domain as gated (Cloudflare Access) noindex staging. - Async jobs: Django-Q2 on existing Redis — move all SMS/email/Brevo sends + appointment reminders to retried tasks; keep OTP verify sync. - Externalize state: managed Postgres + PITR + PgBouncer (transaction pooling), managed Redis; run multiple Gunicorn instances behind the LB (unlocked now that media is on R2). - Security/perf depth: Cloudflare Pro; CSP Report-Only → enforced (self-hosted fonts make it `'self'`); ClamAV upload scanning; Permissions-Policy/CORP headers; PHI-access audit logging in the `compliance` app. - PWA growth: Font Awesome → inline SVG sprite; Pillow WebP responsive pipeline; Android `beforeinstallprompt` button; self-hosted Web Push (VAPID + `pywebpush` , PHI-free payloads) only after install rates justify it.

Pre-Launch Go-Live Checklist

Security - [ ] `DEBUG=0` in production environment - [ ] `python manage.py check --deploy` passes (incl. custom `accounts.E001/E002` feature-flag checks) - [ ] Gunicorn bound to `127.0.0.1` only; origin unreachable except via proxy - [ ] JWT `ROTATE_REFRESH_TOKENS=True` + `BLACKLIST_AFTER_ROTATION=True` ; `token_blacklist` app installed & migrated - [ ] `python-magic` content-type + size allowlist enforced on all uploads - [ ] `MFA_SECRET_KEY` set, ≥ a valid Fernet key, and **backed up securely** (rotating it bricks all staff TOTP) - [ ] Cloudflare SSL = **Full (strict)**, Bot Fight Mode on, Managed Challenge on login/OTP/ `/api/token/` - [ ] gitleaks pre-commit active; confirmed `.env` never committed; one TruffleHog history scan done - [ ] Least-privilege Postgres app role (no SUPERUSER/CREATEDB); separate migration role

Infra / Deploy - [ ] All migration files committed (`.gitignore` line 18 blanket rule removed) - [ ] Media on private R2 (`AWS_DEFAULT_ACL=None` , signed URLs, 5-min expiry, no public r2.dev) - [ ] `migrate` runs in release/pre-deploy phase, never in Gunicorn boot or build - [ ] `pg_dump` taken **before** every migrate; backward-compatible migration discipline confirmed - [ ] `STORAGES["staticfiles"] =`

`CompressedManifestStaticFilesStorage` ; `collectstatic --noinput` succeeds; `whitenoise[brotli]` installed - [ ] Gunicorn `--workers 5 --timeout 30 --max-requests 1000 --max-requests-jitter 100` - [ ] CI green: migrations-check, `check --deploy` , tests, pip-audit, Bandit

Domain / DNS - [ ] DNS on Cloudflare; A/CNAME proxied, DKIM/DMARC/MX grey-cloud - [ ] Brevo DKIM verified + DMARC `p=none` live; `DEFAULT_FROM_EMAIL` on verified domain - [ ] `ALLOWED_HOSTS` + `CSRF_TRUSTED_ORIGINS` set to the live domain (apex + www) - [ ] `X-Robots-Tag: noindex` on authenticated/PHI views and on staging host - [ ] HSTS preload NOT submitted for throwaway test domain (defer to stable brand domain) - [ ] Registrar lock + auto-renew + WHOIS privacy on the real domain (at cutover)

PWA / Perf - [ ] `manifest.webmanifest` (`dir:rtl` , `lang:ar`) + 192/512 + maskable + apple-touch icons + theme-color - [ ] `/sw.js` served at site root with PHI guards (skip `/media/` , `HX-Request` , `Authorization` , non-GET) + `/offline/` - [ ] Cairo/Inter self-hosted (arabic+latin subset, preloaded, `font-display:swap`); Google Fonts CDN removed - [ ] Images have explicit width/height (CLS); JS `defer` red; 0 third-party requests in `<head>`

Monitoring - [ ] `/healthz` + `/readyz` live (DB + Redis), exempt from tenant middleware/auth - [ ] Sentry Developer wired with `send_default_pii=False` + `before_send` PHI scrub - [ ] Better Stack Uptime pinging `/healthz` ; alerts to email/Telegram - [ ] Cloudflare Web Analytics enabled; alert on bulk PHI export events

Legal / Backups - [ ] Arabic + English Privacy Policy, Terms, consent at signup - [ ] Sub-processor list published (R2, Brevo, Twilio, TweetsMS, Sentry, Cloudflare, host) - [ ] Nightly encrypted `pg_dump` →R2 + R2 object versioning enabled - [ ] **One full restore drill completed and logged** (untested backup ≠ backup) - [ ] Staging/test environment seeded with synthetic data only (no real PHI)

---

8. Cross-Section Gaps & Risks

Gaps, Contradictions & Risks Across the Six Sections

- **Cost normalization (resolved):** Section 2 quotes Launch infra at "~$7–9/mo (compute+db+redis+media+CDN)" while Section 3 adds "$0 security" and Section 6 flags SMS separately. These are consistent once you separate **fixed infra (~$6–8/mo)** from **variable SMS (~$60–150/mo)**. The consolidated table makes that split explicit. **Assumption used:** Launch = one Hetzner CX22 + all free tiers; SMS at ~3,000 reminders/mo on MENA rates. The 8 GB ARM CAX21 (~€6.49) is ~$2 more if you want RAM headroom.
- **Render vs VPS ambiguity:** Section 2 recommends a Hetzner CX22 VPS as primary but Section 5's CD examples lean Render/PaaS. Not a contradiction (both are offered as paths), but pick one before wiring CI/CD — the `migrate` -in-release-phase mechanics differ (Render Pre-Deploy command vs VPS SSH script). The checklist assumes whichever you choose, `migrate` must leave the build/boot step.

- **Async-jobs framework naming:** Section 2.3 floats RQ *or* Django-Q2; Sections 6.3 firmly recommends Django-Q2 (built-in scheduler, reuses Redis). Reconciled to **Django-Q2** as the single pick — it best fits the appointment-reminder scheduling need without Celery's overhead.
- **HSTS preload timing is a real trap, well-covered:** preload is already on in settings (1yr + includeSubDomains + preload). Risk: it must NOT be submitted while the throwaway test domain is the live prod host, and every subdomain (incl. staging) must serve HTTPS before submitting the brand apex. Sections 3 and 4 both flag this correctly — just ensure they're sequenced (don't submit until Phase 2 cutover is stable).
- **TweetsMS has no DPA — residual compliance risk:** Section 3.8 notes Palestinian phone numbers + OTP transit TweetsMS likely without a formal DPA. This is an accepted, documented residual risk, not a blocker — record it in the `compliance` sub-processor list and keep SMS payloads PHI-free.
- **CSP + HTMX interaction:** correctly flagged (don't enable `htmx.config.inlineScriptNonce`; keep inline JS out of swapped fragments). This is a "soon after launch" item, not a go-live blocker, but the dependency between CSP enforcement and self-hosting fonts (Phase 1) should be respected.
- **Otherwise coverage is complete:** the three load-bearing repo risks (gitignored migrations at line 18, local `MEDIA_ROOT` at settings.py:254, absent `/healthz`) and the two latent config bugs (spoofable `SECURE_PROXY_SSL_HEADER` at settings.py:324, JWT rotation/blacklist `False` at settings.py:280-281) are all confirmed against the actual repo and consistently addressed across sections.

Appendix A — Hetzner CX22 First-Day Setup (Quick-Start)

A start-to-finish runbook to take a blank Hetzner box to a live HTTPS site. **OS assumed: Ubuntu 24.04 LTS.** Replace `yourdomain.com` and every `CHANGE_ME` placeholder. Steps are ordered — do them top to bottom. Budget ~60–90 minutes the first time.

> This runs everything (Django + Postgres + Redis + Caddy) on the one box, exactly as in §2.2. Media starts on the local disk (which *is* persistent on a VPS, unlike Render) so you can go live today; move it to Cloudflare R2 (§2.4) before you accept real patient uploads — it gives you off-box backup durability and unlocks horizontal scaling.

A.0 Before you touch the server

- [ ] **Hetzner account created and a payment method accepted** (do this *first* — if a MENA card/PayPal is rejected, switch to the Hostinger VPS fallback before wasting setup time).
- [ ] **Migrations committed** to your repo (§2.5) and code pushed to GitHub.
- [ ] An SSH key on your laptop: `ssh-keygen -t ed25519` (add the public key to Hetzner in the next step).
- [ ] A free **Cloudflare account** with the domain ready to add.

- [ ] Generate two production secrets locally and keep them safe: `bash # SECRET_KEY python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())" # MFA_SECRET_KEY (Fernet) — back this up; rotating it bricks all staff TOTP python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`

A.1 Create the server

Hetzner Cloud Console → **Add Server** → Location **Falkenstein or Nuremberg** (Frankfurt region, closest to Gaza) → Image **Ubuntu 24.04** → Type **CX22** (or **CAX21** ARM for 8 GB RAM — recommended for your analytics) → add your SSH key → name it `clink-prod` → **Create**. Copy the **IPv4 address**.

A.2 Base hardening (SSH in as root)

```
ssh root@YOUR_SERVER_IP

apt update && apt -y upgrade

# non-root sudo user
adduser appuser
usermod -aG sudo appuser
rsync --archive --chown=appuser:appuser ~/.ssh /home/appuser   # copy your SSH key over

# firewall: only SSH + HTTP + HTTPS reach the box
ufw allow OpenSSH
ufw allow 80,443/tcp
ufw --force enable

# brute-force protection + automatic security patches
apt -y install fail2ban unattended-upgrades
dpkg-reconfigure -plow unattended-upgrades
```

Disable root + password SSH login — edit `/etc/ssh/sshd_config` so these read:

```
PermitRootLogin no
PasswordAuthentication no
```

Then `systemctl restart ssh` and **reconnect as the new user**: `ssh appuser@YOUR_SERVER_IP` . (Keep the root session open until you confirm the new login works.)

A.3 Swap — essential on the 4 GB CX22

Postgres + Redis + 5 Gunicorn workers + pandas/numpy/scikit-learn can exhaust 4 GB. A swap file prevents the OOM-killer from killing Postgres mid-query:

```
sudo fallocate -l 3G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-swap.conf && sudo sysctl -p /etc/sysctl.d/99-swap.conf
```

(On the 8 GB CAX21 you can skip this or use 2G.)

A.4 Install packages

```
sudo apt -y install python3 python3-venv python3-dev build-essential \
  libpq-dev postgresql redis-server git libmagic1 curl gpg
```

Install **Caddy** (auto-HTTPS reverse proxy):

```
sudo apt -y install debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg --dearmor -o
/usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee
/etc/apt/sources.list.d/caddy-stable.list
sudo apt update && sudo apt -y install caddy
```

> ⚠️ **Requirements gotcha you WILL hit:** `requirements.txt` pins `python-magic-bin==0.4.14`, which is a **Windows/macOS-only** wheel — `pip install` will **fail on Linux**. On the server, install the system `libmagic1` (done above) and use plain `python-magic` instead. Before installing, edit the file on the server (or keep a `requirements-linux.txt`) and change `python-magic-bin==0.4.14` → `python-magic==0.4.27`. This is the one dependency edit the Linux box needs.

A.5 PostgreSQL — database + least-privilege app role

```
sudo -u postgres psql <<'SQL'
CREATE DATABASE clinic_db;
CREATE USER clink_app WITH PASSWORD 'CHANGE_ME_STRONG_DB_PW';
ALTER ROLE clink_app SET client_encoding TO 'utf8';
ALTER ROLE clink_app SET default_transaction_isolation TO 'read committed';
ALTER ROLE clink_app SET timezone TO 'Asia/Gaza';
GRANT ALL PRIVILEGES ON DATABASE clinic_db TO clink_app;
SQL
# PostgreSQL 16 locks down the public schema — grant it explicitly:
sudo -u postgres psql -d clinic_db -c "GRANT ALL ON SCHEMA public TO clink_app;"
```

Postgres stays bound to `localhost` by default — leave it that way (never expose 5432 publicly; UFW already blocks it).

A.6 Redis

Ubuntu's Redis already binds to `127.0.0.1`. Optionally cap memory in `/etc/redis/redis.conf`:

```
maxmemory 256mb
maxmemory-policy noeviction
```

> **Keep `noeviction` (the default), not an LRU policy.** Redis here holds your OTP throttles, brute-force lockouts and MFA limits — an LRU policy could silently evict those security counters under memory pressure. Better to alert on Redis memory (see §5.4) than to let it drop a lockout. Restart: `sudo systemctl restart redis-server`.

A.7 Deploy the app

```
sudo mkdir -p /srv/clink && sudo chown appuser:appuser /srv/clink
cd /srv/clink
git clone https://github.com/YOU/your-repo.git .
python3 -m venv venv && source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt        # after the python-magic-bin → python-magic edit (A.4)
```

Create `/srv/clink/.env` (then `chmod 600 .env`) with **production** values:

```
SECRET_KEY=<the get_random_secret_key value>
DEBUG=0
ALLOWED_HOSTS=yourdomain.com,www.yourdomain.com
CSRF_TRUSTED_ORIGINS=https://yourdomain.com,https://www.yourdomain.com

DB_NAME=clinic_db
DB_USER=clink_app
DB_PASSWORD=CHANGE_ME_STRONG_DB_PW
DB_HOST=localhost
DB_PORT=5432

REDIS_URL=redis://127.0.0.1:6379/1

MFA_SECRET_KEY=<the Fernet key>
ENFORCE_PHONE_VERIFICATION=1
ENFORCE_OTP_LIMITS=1

SMS_PROVIDER=TWEETSMS
TWEETSMS_API_KEY=CHANGE_ME
TWEETSMS_SENDER=CHANGE_ME
BREVO_API_KEY=CHANGE_ME

# Cloudflare R2 (uncomment after wiring django-storages per §2.4)
# R2_ENDPOINT=https://<acct>.r2.cloudflarestorage.com
# R2_ACCESS_KEY_ID=CHANGE_ME
# R2_SECRET_ACCESS_KEY=CHANGE_ME


chmod 600 .env
python manage.py check --deploy        # MUST pass with DEBUG=0 (incl. accounts.E001/E002)
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py createsuperuser
```

A.8 Gunicorn as a systemd service

Bind to `127.0.0.1` only (so the unconditional `SECURE_PROXY_SSL_HEADER` at `settings.py:324` can't be spoofed from outside). Create `/etc/systemd/system/gunicorn.service`:

```
[Unit]
Description=clink gunicorn
After=network.target postgresql.service redis-server.service

[Service]
User=appuser
Group=appuser
WorkingDirectory=/srv/clink
EnvironmentFile=/srv/clink/.env
ExecStart=/srv/clink/venv/bin/gunicorn clinic_website.wsgi:application \
  --workers 5 --timeout 30 --max-requests 1000 --max-requests-jitter 100 \
  --bind 127.0.0.1:8000
Restart=always

[Install]
WantedBy=multi-user.target


sudo systemctl daemon-reload
sudo systemctl enable --now gunicorn
sudo systemctl status gunicorn        # should be active (running)
```

(Prefer a unix socket as in §2.2? Use `--bind unix:/run/clink/gunicorn.sock` plus `RuntimeDirectory=clink` and point Caddy at the socket. TCP-on-localhost is simpler and just as private behind UFW.)

A.9 Caddy reverse proxy

Replace `/etc/caddy/Caddyfile` with:

```
yourdomain.com, www.yourdomain.com {
    encode zstd gzip
    reverse_proxy 127.0.0.1:8000 {
        header_up X-Forwarded-Proto https
    }
}


sudo systemctl reload caddy
```

Caddy auto-obtains + auto-renews a free Let's Encrypt certificate **once DNS points at the box** (next step) and handles the HTTP→HTTPS redirect for you.

A.10 DNS + Cloudflare — mind the certificate ordering

1. Add the domain to **Cloudflare** (free plan). Cloudflare shows you **2 nameservers**.
2. **Hostinger → Domains → your domain → DNS / Nameservers → Change nameservers → Use custom nameservers** → paste Cloudflare's two NS values → save. (Propagation: minutes to a few hours.)
3. In **Cloudflare → DNS**, add two records, **set to DNS-only (grey cloud) for now**: - `A` · `@` · `YOUR_SERVER_IP` · **DNS only** - `A` · `www` · `YOUR_SERVER_IP` · **DNS only**
4. Wait until `https://yourdomain.com` loads with a valid padlock. **Grey-cloud is required here** so Caddy's Let's Encrypt challenge reaches your server directly.

5. Once HTTPS works, flip both records to **Proxied (orange cloud)** and set **Cloudflare → SSL/TLS → Overview → Full (strict)**. Now Cloudflare's CDN + WAF + DDoS sit in front, and Caddy's real cert satisfies Full (strict) with no redirect loop.

> **For a clean permanent setup** (so renewals never depend on grey-clouding): install a **Cloudflare Origin Certificate** (15-year) into Caddy and stay orange-clouded, *or* use Caddy's Cloudflare-DNS plugin for the DNS-01 challenge. Either removes the manual dance — do it once you're past launch.

A.11 Go-live verification

```
curl -sI https://yourdomain.com | grep -i 'strict-transport-security'   # HSTS header present
sudo systemctl status gunicorn caddy postgresql redis-server            # all active
cd /srv/clink && source venv/bin/activate && python manage.py check --deploy   # 0 issues
```

Then in a browser: padlock is valid, `DEBUG=0` (no debug page on a bad URL), login + OTP flow works, an HTMX page (e.g. doctor workspace notes/orders) swaps correctly. Add the `/healthz` endpoint from §5.2 and point Better Stack at it.

A.12 Future deploys — one script

`/srv/clink/deploy.sh` (`chmod +x`), run after each merge to main:

```
#!/usr/bin/env bash
set -euo pipefail
cd /srv/clink
./backup.sh                              # snapshot DB BEFORE migrating (A.13)
git pull
source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate                 # release phase — before restart
python manage.py collectstatic --noinput
sudo systemctl restart gunicorn
echo "Deployed $(git rev-parse --short HEAD)"
```

Order is always **backup → pull → migrate → collectstatic → restart**.

A.13 Daily encrypted backup → R2

Install `rclone` and configure an `r2:` remote (`rclone config` , S3-compatible, your R2 keys). Then `/srv/clink/backup.sh` :

```
#!/usr/bin/env bash
set -euo pipefail
STAMP=$(date +%F-%H%M)
pg_dump clinic_db | gzip | \
  gpg --symmetric --batch --passphrase-file /srv/clink/.bkpass \
  > /tmp/clink-$STAMP.sql.gz.gpg
rclone copy /tmp/clink-$STAMP.sql.gz.gpg r2:clink-backups/
find /tmp -name 'clink-*.sql.gz.gpg' -mmin +120 -delete
```

Schedule it nightly: `crontab -e` → `0 2 * * * /srv/clink/backup.sh`. **Then actually restore one dump into a scratch database** — an untested backup is not a backup (§6.2).

This plan was generated from a multi-agent architecture review of the current codebase. No application code was modified in producing it. Figures are 2026 estimates — verify with each provider before committing.